TRACER: Signature-based Static Analysis for Detecting Recurring Vulnerabilities

Wooseok Kang
KAIST
Korea
kangwoosukeq@kaist.ac.kr

Byoungho Son*
POSTECH
Korea
byhoson@postech.ac.kr

Kihong Heo
KAIST
Korea
kihong.heo@kaist.ac.kr

ABSTRACT

Similar software vulnerabilities recur because developers reuse existing vulnerable code, or make similar mistakes when implementing the same logic. Recently, various analysis techniques have been proposed to find *syntactically* recurring vulnerabilities via code reuse. However, limited attention has been devoted to *semantically* recurring ones that share the same vulnerable behavior in different code structures. In this paper, we present a general analysis framework, called TRACER, for detecting such recurring vulnerabilities. The main idea is to represent vulnerability signatures as traces over interprocedural data dependencies. TRACER is based on a taint analysis that can detect various types of vulnerabilities. For a given set of *known* vulnerabilities, the taint analysis extracts vulnerable traces and establishes a signature database of them. When a *new unseen* program is analyzed, TRACER compares all potentially vulnerable traces reported by the analysis with the known vulnerability signatures. Then, TRACER reports a list of potential vulnerabilities ranked by the similarity score. We evaluate TRACER on 273 Debian packages in C/C++. Our experiment results demonstrate that TRACER is able to find 112 previously unknown vulnerabilities with 6 CVE identifiers assigned.

CCS CONCEPTS

• **Security and privacy → Software security engineering**; **Software security engineering**.

KEYWORDS

software security, program analysis, software engineering

ACM Reference Format:
Wooseok Kang, Byoungho Son, and Kihong Heo. 2022. TRACER: Signature-based Static Analysis for Detecting Recurring Vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22), November 7–11, 2022, Los Angeles, CA, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3548606.3560664

*This work was done during internship at KAIST.

CCS '22, November 7–11, 2022, Los Angeles, CA, USA

ACM ISBN 978-1-4503-9450-5/22/11…$15.00
https://doi.org/10.1145/3548606.3560664

1 INTRODUCTION

Similar software vulnerabilities recur over time even across programs. One of the well-known reasons is the prevalence of code reuse [23, 26, 33, 34, 47] that can lead to the spread of security vulnerabilities in the reused code. In addition to such *syntactic* recurrences, *semantically* similar vulnerabilities frequently recur in unrelated codebases that are independently developed. One of the reasons is that developers often make similar mistakes when implementing the same standard concepts such as mathematical formulas, laws of physics, protocols, or language interpreters [37, 42]. Another reason is common misconceptions due to complicated low-level semantics of programming languages such as undefined behaviors in C [14]. They can induce developers to write incorrect code with similar error patterns. According to a recent report from Google, 6 out of 24 0-day vulnerabilities in 2020 were actually variants of previously seen ones [42].

Although researchers have developed many successful techniques to detect recurring security vulnerabilities, existing approaches have limitations in several aspects. Approaches based on code similarity [12, 15, 23, 26, 29, 38, 47] aim at detecting recurring vulnerabilities via code reuse. They generate signatures of known vulnerabilities within a pre-defined boundary (e.g., file or function) and compare syntactic patterns in a new program with the signatures. These approaches are highly precise, scalable and general as their approaches are based on syntactic matching. However, they are usually unable to detect variants of known vulnerabilities with completely different syntactic structures but with the same root causes. On the other hand, pattern-based static analyses [2, 3, 18] estimate the semantics of target programs as well as consider their syntactic patterns. This in turn enables the analyzer to detect vulnerabilities that have similar syntactic and semantic characteristics of programs with known vulnerabilities. However, designing such analyses requires static analysis expertise and incurs a nontrivial engineering burden.

To address this problem, we set out to build an effective *software immune system* against recurring vulnerabilities. We identified the following criteria to be satisfied for such a system:

- *Accuracy*: Does the system accurately report potential vulnerabilities with a low false positive rate?
- *Robustness*: Is the system able to find variants of vulnerabilities that have the same root cause?
- *Generality*: Is the system applicable to a wide range of security bugs?
- *Scalability*: Is the system applicable to large programs?
- *Usability*: Does the system provide easily interpretable reports?

In this paper, we present a signature-based static analysis for detecting recurring vulnerabilities, TRACER, that is designed to satisfy

Tracer：基于签名的静态分析，用于检测重复出现的漏洞

姜宇硕
韩国科学技术院
韩国
kangwoosukeq@kaist.ac.kr

孙炳浩
邮政
韩国
byhoson@postech.ac.kr

猪基洪
韩国科学技术院
韩国
kihong.heo@kaist.ac.kr

抽象的

类似的软件漏洞之所以会反复出现，是因为开发人员重用了现有的易受攻击的代码，或者在实现相同逻辑时犯了类似的错误。近年来，各种分析技术已被提出，旨在通过代码重用发现语法上重复出现的漏洞。然而，对于在不同代码结构中具有相同漏洞行为的语义上重复出现的漏洞，关注度却有限。本文提出了一个通用的分析框架 Tracer，用于检测此类重复出现的漏洞。其主要思想是将漏洞签名表示为过程间数据依赖关系的痕迹。Tracer 基于污点分析，可以检测各种类型的漏洞。对于给定的一组已知漏洞，污点分析会提取漏洞痕迹并建立它们的签名数据库。当分析一个新的未知程序时，Tracer 会将分析报告的所有潜在漏洞痕迹与已知漏洞签名进行比较。然后，Tracer 会报告一个按相似度得分排序的潜在漏洞列表。我们在 273 个 Debian C/C++ 软件包上对 Tracer 进行了评估。我们的实验结果表明，Tracer 能够找到 112 个以前未知的漏洞，并分配了 6 个 CVE 标识符。

CCS 概念

• 安全和隐私 → 软件安全工程；软件洁具安全工程。

关键词

软件安全、程序分析、软件工程

ACM 参考格式：
Wooseok Kang、Byoungho Son 和 Kihong Heo。2022 年。Tracer：基于签名的静态分析，用于检测重复出现的漏洞。会议论文集
2022 年 ACM SIGSAC 计算机与通信安全会议 (CCS '22) 论文集，2022 年 11 月 7 日至 11 日，美国加利福尼亚州洛杉矶。ACM，美国纽约州纽约市，共 14 页。https://doi.org/10.1145/3548606.3560664

∗ 这项工作是在 KAIST 实习期间完成的。

CCS '22，2022 年 11 月 7 日至 11 日，美国加利福尼亚州洛杉矶© 2022 计算机协会。
ACM ISBN 978-1-4503-9450-5/22/11。。。
15.00 美元
https://doi.org/10.1145/3548606.3560664

1 引言

类似的软件漏洞甚至会在不同程序之间重复出现。其中一个众所周知的原因是代码重用盛行 [23、26、33、34、47]，这会导致安全漏洞在重用代码中蔓延。除了这种语法重现之外，语义上相似的漏洞还经常在独立开发的不相关代码库中重复出现。原因之一是开发人员在实现相同的标准概念（例如数学公式、物理定律、协议或语言解释器）时经常会犯类似的错误 [37、42]。另一个原因是由于编程语言复杂的低级语义（例如 C 中的未定义行为）导致的常见误解 [14]。它们会诱使开发人员编写具有相似错误模式的错误代码。根据谷歌最近的一份报告，2020 年 24 个 0-day 漏洞中有 6 个实际上是以前见过的漏洞的变种 [42]。

尽管研究人员已经开发出许多成功的技术来检测重复出现的安全漏洞，但现有方法在多个方面仍存在局限性。基于代码相似性的方法 [12, 15 , 23 , 26 , 29 , 38, 47] 旨在通过代码重用来检测重复出现的漏洞。它们在预定义的边界（例如文件或函数）内生成已知漏洞的签名，并将新程序中的句法模式与签名进行比较。由于这些方法基于句法匹配，因此具有高度的精确性、可扩展性和通用性。然而，它们通常无法检测到具有完全不同句法结构但具有相同根本原因的已知漏洞的变体。另一方面，基于模式的静态分析 [2, 3, 18] 会估计目标程序的语义并考虑其句法模式。这反过来又使分析器能够检测到与已知漏洞程序具有相似句法和语义特征的漏洞。然而，设计此类分析需要静态分析专业知识，并且会带来不小的工程负担。

为了解决这个问题，我们着手构建一个有效的软件免疫系统，以抵御反复出现的漏洞。我们确定了该系统需要满足以下标准：

- 准确性：系统是否准确报告潜在漏洞且误报率较低？
- 稳健性：系统是否能够找到具有相同根本原因的漏洞变体？
- 通用性：该系统是否适用于广泛的安全漏洞？
- 可扩展性：该系统是否适用于大型程序？
- 可用性：系统是否提供易于理解的报告？本文提出了一种基于签名的静态分析方法，用于检测重复出现的漏洞，即 Tracer，旨在满足以下要求：

the above criteria. The key idea is to represent vulnerability signatures as traces over interprocedural data dependencies. Tracer is based on a general taint analysis that aims at a variety of security vulnerabilities such as integer overflow/underflow, format string, buffer overflow, command injection, etc. The analyzer detects potentially vulnerable data flows from untrusted inputs (so called, *source*) to security-sensitive functions (so called, *sink*). We run the static analyzer on a codebase with known vulnerabilities and identify the actual vulnerabilities in the analysis results. Next, Tracer extracts traces on the data dependency relations of the vulnerabilities from the source points to the sink points. The traces are encoded as feature vectors that form the signatures of the vulnerabilities. Once a new program is analyzed, Tracer extracts traces of all the reported alarms in the program, and derives their feature vectors in the same manner. Then, Tracer compares the feature vectors of the alarms with those of the known vulnerable traces using a typical similarity measure such as cosine similarity. Finally, Tracer provides a list of alarms sorted by similarity.

We implemented Tracer based on Facebook's Infer analyzer [5] and demonstrated the effectiveness on a suite of Debian packages written in C/C++. According to our experimental results on 273 Debian packages, Tracer discovered 112 recurring vulnerabilities that are similar to known CVEs, vulnerability examples in Juliet test suite [4], and sample code in online tutorials for secure coding.

This paper makes the following contributions:

- We propose a general analysis framework, Tracer, for detecting semantically recurring vulnerabilities. Tracer is applicable to a wide range of vulnerabilities.
- We present a trace-based method for computing the similarity of vulnerabilities. Our method is based on data dependencies of alarms reported by a general taint analysis.
- We evaluate the effectiveness of Tracer on 273 Debian packages. We found 112 vulnerabilities with 6 CVE identifiers assigned.

2 OVERVIEW

2.1 Motivating Examples

We illustrate our approach with the programs with security vulnerabilities in Figure 1. All three programs have similar issues related to a certain kind of security vulnerability: overflowed integers can be used as the size argument of memory allocation functions (e.g., `malloc`). Such integer overflows cause the program to unintentionally allocate small memory chunks, potentially leading to buffer overflows.

Figure 1(a) shows the vulnerability in an image processing tool gimp reported in 2009. The program reads a byte string from a given file (line 10), transforms the string into an integer (line 12). Since this value depends on the contents of the input file, the integer can be arbitrarily large. The integer value at line 13 can also become arbitrarily large because of the same reason. Then, the program multiplies the integers, leading to an integer overflow (line 14). Finally, the overflowed integer (`rowbytes`) is passed to the function `ReadImage` and used as an argument of `malloc` (line 21). Notice that the size of the allocated buffer can be much smaller than what the developer expected. Therefore, potential buffer overflows can happen when the buffer is used to store the data of the input file afterward.

```
1  gint32 ToL(guchar *puffer) {
2    return (puffer[0] | puffer[1] << 8 | puffer[2] << 16 | puffer[3] << 24);
3  }
4  gint16 ToS(guchar *puffer) { return (puffer[0] | puffer[1] << 8); }
5
6  gint32 ReadBMP(gchar *name) {
7    FILE *fd = fopen(name, "rb");
8    if (!fd) return -1;
9    // Read from a file
10   if (fread(buffer, Bitmap_File_Head.biSize - 4, fd) != 0)
11     return -1;
12   Bitmap_Head.biWidth = ToL(& buffer[0x00]);
13   Bitmap_Head.biBitCnt = ToS(& buffer[0x0A]);
14   rowbytes = ((Bitmap_Head.biWidth * Bitmap_Head.biBitCnt - 1) / 32) * 4 + 4;
15   image_ID = ReadImage(rowbytes);
16     ...
17 }
18
19 gint32 ReadImage(gint rowbytes) {
20     /* memory allocation with an overflowed size */
21     guchar *buffer = malloc(rowbytes);
22     /* uses of buffer */
23 }
```

(a) gimp-2.6.7 (CVE-2009-1570)

```
1  long ToL(unsigned char *puffer) {
2    return (puffer[0] | puffer[1] << 8 | puffer[2] << 16 | puffer[3] << 24);
3  }
4  short ToS(unsigned char *puffer) { return (puffer[0] | puffer[1] << 8); }
5
6  bitmap_type bmp_load_image(FILE *fd) {
7    if (fread(buffer, 18, fd) || (strncmp((const char *)buffer, "BM", 2)))
8      FATALP("BMP:_not_a_valid_BMP_file");
9    // Read from a file
10   if (fread(buffer, Bitmap_File_Head.biSize - 4, fd) != 0)
11     FATALP("BMP:_Error_reading_BMP_file_header_#3");
12   Bitmap_Head.biWidth = ToL(&buffer[0x00]);
13   Bitmap_Head.biBitCnt = ToS(&buffer[0x0A]);
14   rowbytes = ((Bitmap_Head.biWidth * Bitmap_Head.biBitCnt - 1) / 32) * 4 + 4;
15   image.bitmap = ReadImage(rowbytes);
16   ...
17 }
18
19 unsigned char *ReadImage(int rowbytes) {
20   /* memory allocation with an overflowed size */
21   unsigned char *buffer = (unsigned char *) new char[rowbytes];
22   /* uses of buffer */
23 }
```

(b) sam2p-0.49.4 (CVE-2017-16663)

```
1  XcursorBool _XcursorReadUInt(XcursorFile *file, XcursorUInt *u) {
2    unsigned char bytes[4];
3    if ((*file->read)(file, bytes, 4) != 4)   // Read from a file
4      return XcursorFalse;
5    *u = (bytes[0] | (bytes[1] << 8) | (bytes[2] << 16) | (bytes[3] << 24));
6    return XcursorTrue;
7  }
8
9  XcursorImage *_XcursorReadImage(XcursorFile *file) {
10   XcursorImage head;
11   XcursorImage *image;
12   if (!_XcursorReadUInt(file, &head.width)) return NULL;
13   if (!_XcursorReadUInt(file, &head.height)) return NULL;
14   image = XcursorImageCreate(head.width, head.height);
15   ...
16 }
17
18 XcursorImage *XcursorImageCreate(int width, int height) {
19   XcursorImage *image;
20   /* memory allocation with an overflowed size */
21   image = malloc(sizeof(XcursorImage) + width * height * sizeof(XcursorPixel));
22   /* initialize struct image */
23   return image;
24 }
```

(c) libXcursor-1.1.14 (CVE-2017-16612)

Figure 1: Examples code excerpted from similar vulnerabilities from different programs.

上述标准。其核心思想是将漏洞特征表示为过程间数据依赖关系的轨迹。Tracer 基于通用污点分析，旨在解决各种安全漏洞，例如整数溢出/下溢、格式字符串、缓冲区溢出、命令注入等。该分析器检测从不受信任的输入（称为“源”）到安全敏感函数（称为“接收器”）的潜在漏洞数据流。我们在包含已知漏洞的代码库上运行静态分析器，并在分析结果中识别出实际漏洞。接下来，Tracer 提取漏洞从源点到接收器的数据依赖关系轨迹。这些轨迹被编码为特征向量，构成漏洞的特征。分析新程序后，Tracer 提取程序中所有已报告警报的轨迹，并以相同的方式导出它们的特征向量。然后，Tracer 使用典型的相似性度量（例如余弦相似度）将警报的特征向量与已知漏洞轨迹的特征向量进行比较。最后，Tracer 提供了按相似性排序的警报列表。

我们基于 Facebook 的 Infer 分析器 [5] 实现了 Tracer，并在一系列用 C/C++ 编写的 Debian 软件包上证明了其有效性。根据我们在 273 个 Debian 软件包上的实验结果，Tracer 发现了 112 个与已知 CVE、Juliet 测试套件 [4] 中的漏洞示例以及在线安全编码教程中的示例代码类似的重复漏洞。

本文做出了以下贡献：

- 我们提出了一个通用的分析框架 Tracer，用于检测语义重复的漏洞。Tracer 适用于各种类型的漏洞。
- 我们提出了一种基于追踪的漏洞相似度计算方法。该方法基于通用污点分析报告的警报数据依赖关系。
- 我们对 273 个 Debian 软件包评估了 Tracer 的有效性。我们发现了 112 个漏洞，并分配了 6 个 CVE 标识符。

2 概述

2.1 激励示例

我们用图 1 中存在安全漏洞的程序来说明我们的方法。这三个程序都存在类似的安全漏洞：溢出的整数可以用作内存分配函数（例如 malloc）的大小参数。此类整数溢出会导致程序无意中分配较小的内存块，从而可能导致缓冲区溢出。

图 1(a) 展示了 2009 年报告的图像处理工具 gimp 中的漏洞。该程序从给定文件读取一个字节字符串（第 10 行），并将该字符串转换为整数（第 12 行）。由于该值取决于输入文件的内容，因此该整数可以任意大。由于同样的原因，第 13 行的整数值也可以任意大。然后，程序将这些整数相乘，导致整数溢出（第 14 行）。最后，溢出的整数（rowbytes）被传递给函数 ReadImage 并用作 malloc 的参数（第 21 行）。请注意，分配的缓冲区大小可能比开发人员预期的要小得多。因此，当缓冲区随后用于存储输入文件的数据时，可能会发生潜在的缓冲区溢出。

```
1 gint32 ToL (guchar *puffer) {
2         返回 (puffer[0] | puffer[1] << 8 | puffer[2] << 16 | puffer[3] << 24) ;
3 }
4 gint16 ToS(guchar *puffer) { return (puffer[0] | puffer[1] << 8); }
5
6 gint32 ReadBMP(gchar *name) {
7 文件*fd = fopen(名称, “rb”);
8 如果 (! fd) 返回-1;
9 // 从文件读取
10 如果 (fread (缓冲区, Bitmap_File_Head.biSize - 4, fd) ! = 0)
11 返回-1;
12 Bitmap_Head.biWidth = ToL(&缓冲区[0x00]);
13 Bitmap_Head.biBitCnt = ToS(&缓冲区[0x0A]);
14 行字节 = ((Bitmap_Head.biWidth * Bitmap_Head.biBitCnt - 1) / 32) * 4 + 4;
15 image_ID = 读取图像(rowbytes);
16 ...
17 }
18
19 gint32 读取图像 (gint rowbytes) {
20        /* 具有溢出大小的内存分配 */
21         guchar *缓冲区=malloc (rowbytes) ;
22       /* 缓冲区的使用 */
23 }
```

(一) gimp-2.6.7 (CVE-2009-1570)

```
1 长 ToL（无符号字符 *puffer）{
2 返回 (puffer[0] | puffer[1] << 8 | puffer[2] << 16 | puffer[3] << 24) ;
3 }
4 short ToS(unsigned char *puffer) { return (puffer[0] | puffer[1] << 8); }
5
6 位图类型 bmp_load_image (文件 *fd) {
7 如果 (fread (缓冲区, 18, fd) || (strncmp ( (const char *) 缓冲区, “BM”, 2) ) )
8 FATALP("BMP:不是一个有效的 BMP 文件");
9 // 从文件读取
10 如果 (fread (缓冲区, Bitmap_File_Head.biSize - 4, fd) ! = 0)
11 FATALP("BMP:读取 BMP 文件头时出错 #3");
12 Bitmap_Head.biWidth = ToL(&buffer[0x00]);
13 Bitmap_Head.biBitCnt = ToS(&buffer[0x0A]);
14 行字节 = ((Bitmap_Head.biWidth * Bitmap_Head.biBitCnt - 1) / 32) * 4 + 4;
15 图像.位图 = 读取图像(rowbytes);
16 ...
17 }
18
19 无符号字符 *ReadImage(int rowbytes) {
20 /* 具有溢出大小的内存分配 */
21 无符号字符*缓冲区=（无符号字符*）新字符[rowbytes];
22 /* 缓冲区的使用 */
23 }
```

(b) sam2p-0.49.4 (CVE-2017-16663)

```
1 XcursorBool _XcursorReadUInt(XcursorFile *file, XcursorUInt *u) {
2 个无符号字符字节[4];
3 if ((*file->read)(file, bytes, 4) != 4) // 从文件读取
4 返回 XcursorFalse;
5 *u = (字节[0] | (字节[1] << 8) | (字节[2] << 16) | (字节[3] << 24));
6 返回 XcursorTrue;
7 }
8
9 XcursorImage *_XcursorReadImage (XcursorFile *文件) {
10 XcursorImage 头;
11 XcursorImage *图像;
12 如果 (! _XcursorReadUInt (file, &head.width) ) 返回 NULL;
13 如果 (! _XcursorReadUInt (file, &head.height) ) 返回 NULL;
14 图像 = XcursorImageCreate(head.width, head.height);
15 ...
16 }
17
18 XcursorImage *XcursorImageCreate(int 宽度, int 高度) {
19 XcursorImage *图像;
20 /* 具有溢出大小的内存分配 */
21 图像 = malloc(sizeof(XcursorImage) + 宽度 * 高度 * sizeof(XcursorPixel));
22 /* 初始化结构图像 */
23 返回图像;
24 }
```

(c) libXcursor-1.1.14 (CVE-2017-16612)

图 1：从不同程序的类似漏洞中摘录的示例代码。

2.1就是介绍了几个漏洞例子1

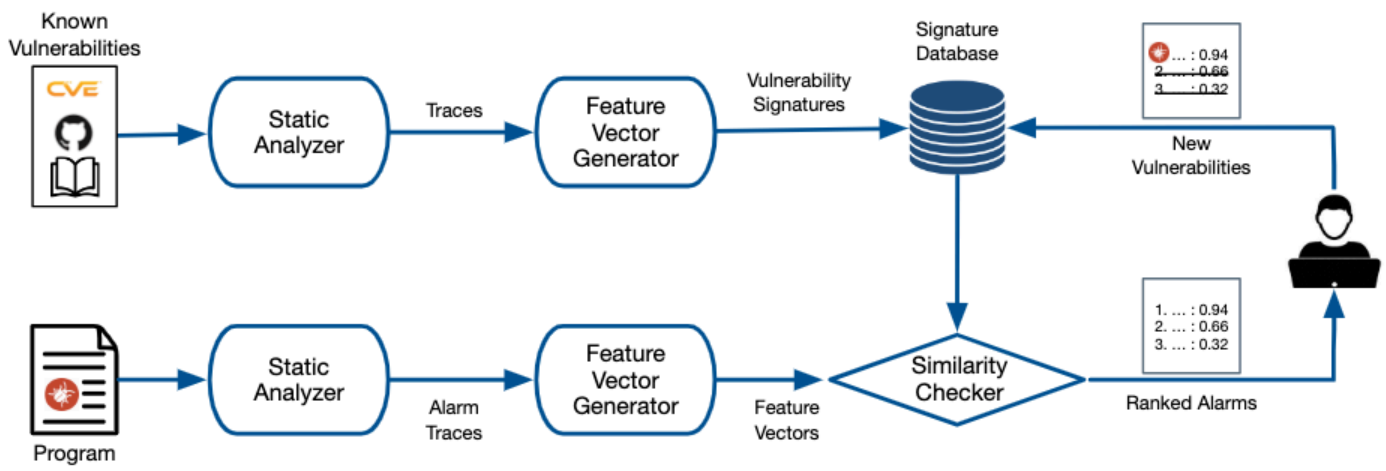


Figure 2: System overview of TRACER

After 8 years, a similar vulnerability was found in another program, sam2p depicted in Figure 1(b). sam2p is also an image processing tool, so it has a similar piece of code that reads a BMP file. Because of exactly the same reason as gimp, this program is also vulnerable. Notice that the code snippet is quite similar to that of gimp. Conceptually, existing methods based on code clone detection may help catch such recurring vulnerabilities given the vulnerability in gimp as a *signature*. However, it is sometimes challenging in practice. Clone-based approaches typically compare two pieces of code within a pre-defined syntactic boundary (e.g., functions or blocks). This in turn hinders vulnerability detection when vulnerable behavior involves multiple functions as in the examples. State-of-the-art tools [26, 47] heuristically choose a vulnerability signature function that contains the patches of the known vulnerability (`ReadBMP` in the gimp case). However, this is still fragile if the functions are large and contain considerable syntactic differences. For example, `ReadBMP` in gimp consists of 382 lines while `bmp_load_image` in samp2p has only 151 lines. Although the essence of the vulnerability is the same, they have many discrepancies in the other parts. For example, lines 7–8 in the two programs are completely different, and sam2p, which is a C++ program, uses `new` rather than `malloc`.

Moreover, recurring vulnerabilities are not always induced by code clones. Developers often make similar mistakes when they write programs that have typical or standard behavior both at a low level (e.g., reading data from files or allocating heap memory blocks) and at a high-level (e.g., calculating the area of a square or processing an image file). An example from libXcursor is shown in Figure 1(c). Similar to the previous examples, the program reads data from an input file (line 3), converts the input byte string to an integer (line 5), and computes the multiplication of two arbitrary large integers (line 21). The multiplication also leads to an integer overflow at the same line that can cause buffer overflows afterward. Notice that the root cause of the vulnerability is the same as the other examples. However, libXcursor has completely different syntactic structures. For example, libXcursor uses an indirect call to `fread` at line 3 while the other programs directly call the function.

Existing approaches are not appropriate to detect such *semantically* recurring vulnerabilities. Clone-based approaches [26, 47] are not effective to detect this vulnerability, given the vulnerability in gimp or sam2p as a signature. While the essence of the vulnerability is still the same, the different code structure of libXcursor fundamentally hinders the detectability of the tools. Static bug-finding tools that aim at general integer overflows may detect this vulnerability but also can incur many false positives. One can also design a specialized static analysis dedicated to each pattern. However, it would impose a high engineering burden while producing sub-optimal solutions. For example, the TaintedAllocationSize checker from Github's CodeQL [8], which is a state-of-the-art pattern-based analyzer, does not detect the particular vulnerabilities in Figure 1.

2.2 Our Approach

Now, we introduce how TRACER can detect recurring vulnerabilities. Our approach is shown in Figure 2. In the rest of this section, we explain the procedure of each component of TRACER and show the vulnerabilities in sam2p and libXcursor can be accurately detected by TRACER given the one in gimp as a signature.

2.2.1 Taint Analysis. TRACER is based on a generic taint analysis that can be instantiated to bug detectors for various types of security vulnerabilities. The analysis computes potential data flows from untrusted inputs (sources) to sensitive functions (sinks) with a simple abstract domain for tainted values: $\mathbb{T} = \{\bot_t, \top_t\}$ where each element denotes that the value is not tainted ($\bot_t$) and may be tainted ($\top_t$). For example, in Figure 1(a), the malicious data flow from `fread` to `malloc` is detected by the analyzer.

One may elaborate the analysis with other abstract domains along with the basic taint domain for a more accurate analysis. In our implementation, we have a simple abstract domain $\bar{\mathbb{I}} = \{\bot_o, \top_o\}$ for estimating whether an integer value is potentially overflowed ($\top_o$) or not ($\bot_o$). For example, an untrusted input value is initially tainted ($\top_t$) but not overflowed ($\bot_o$). Once the value is used as an operand of an operator that can potentially introduce integer overflow (e.g., +, <<), the result becomes tainted ($\top_t$) and overflowed ($\top_o$). For the `malloc` case, our analyzer raises an alarm only when the abstract value of the argument is both tainted ($\top_t$) and overflowed ($\top_o$). By doing so, we do not report trivial false alarms while efficiently computing malicious data flows. The details of our implementation are described in Section 4.

2.2.2 Traces on Data Dependency Graphs. We run the taint analysis on a given set of programs whose vulnerabilities are already known. For each known vulnerability, TRACER extracts vulnerable traces from the source and sink points based on the static analysis result. To filter out statements that are irrelevant to the vulnerability as much as possible, we derive vulnerable traces on data

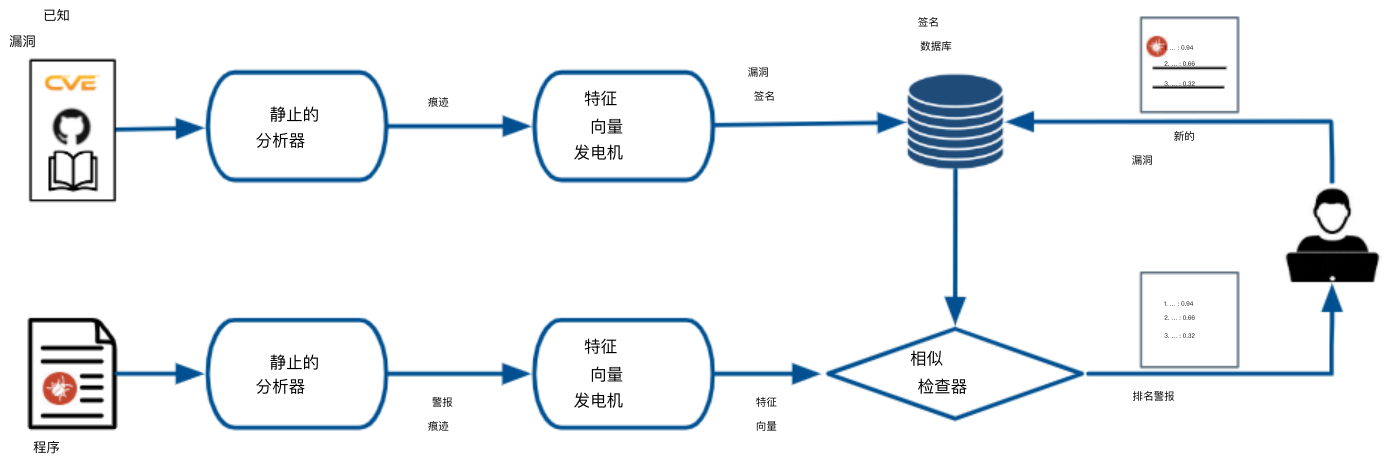


图 2：Tracer 系统概览

一些漏洞反复出现，一方面代码克隆、另一方面一些语义错误总是犯

八年后，另一个程序 sam2p（如图 1(b) 所示）中也发现了类似的漏洞。sam2p 也是一个图像处理工具，因此它包含一段类似的代码来读取 BMP 文件。由于与 gimp 完全相同的原因，该程序也存在漏洞。请注意，该代码片段与 gimp 的代码片段非常相似。从概念上讲，现有的基于代码克隆检测的方法，如果将 gimp 中的漏洞作为签名，或许有助于捕获此类反复出现的漏洞。然而，实践中有时却充满挑战。基于克隆的方法通常会比较预定义语法边界（例如，函数或代码块）内的两段代码。当漏洞行为涉及多个函数（如示例中所示）时，这反而会阻碍漏洞检测。最先进的工具 [26, 47] 会启发式地选择包含已知漏洞补丁的漏洞签名函数（在 gimp 中为 ReadBMP）。然而，如果函数很大且语法差异很大，这种方法仍然很脆弱。例如，gimp 中的 ReadBMP 函数有 382 行，而 samp2p 中的 bmp_load_image 函数只有 151 行。虽然漏洞的本质相同，但在其他部分存在诸多差异。例如，两个程序的第 7–8 行代码完全不同，并且 samp2p 作为 C++ 程序，使用了 new 而不是 malloc。

此外，反复出现的漏洞并非总是由代码克隆引起的。开发人员在编写具有典型或标准行为的程序时，无论是在低级（例如，从文件读取数据或分配堆内存块）还是高级（例如，计算正方形面积或处理图像文件），都经常会犯类似的错误。图 1(c) 显示了 libXcursor 的一个示例。与前面的示例类似，该程序从输入文件读取数据（第 3 行），将输入字节字符串转换为整数（第 5 行），并计算两个任意大整数的乘积（第 21 行）。该乘法还导致同一行发生整数溢出，这可能导致随后的缓冲区溢出。请注意，该漏洞的根本原因与其他示例相同。但是，libXcursor 具有完全不同的语法结构。例如，libXcursor 在第 3 行使用间接调用 fread 函数，而其他程序则直接调用该函数。

现有方法不适合检测此类语义上重复出现的漏洞。由于 gimp 或 sam2p 中的漏洞具有签名，基于克隆的方法 [26 , 47] 无法有效检测此漏洞。虽然漏洞的本质相同，但 libXcursor 的不同代码结构从根本上阻碍了工具的可检测性。

针对一般整数溢出的静态漏洞查找工具或许能够检测到此漏洞，但也可能产生许多误报。此外，还可以设计专门针对每种模式的静态分析工具。然而，这会造成较高的工程负担，并且会产生次优的解决方案。例如，Github CodeQL [8] 中的 TaintedAllocationSize 检查器，虽然是一款最先进的基于模式的分析器，却无法检测到图 1 中的特定漏洞。

2.2 我们的方法

现在，我们介绍 Tracer 如何检测重复出现的漏洞。我们的方法如图 2 所示。在本节的剩余部分，我们将解释 Tracer 各个组件的工作流程，并展示如何利用 gimp 中的漏洞作为签名，准确检测出 sam2p 和 libXcursor 中的漏洞。

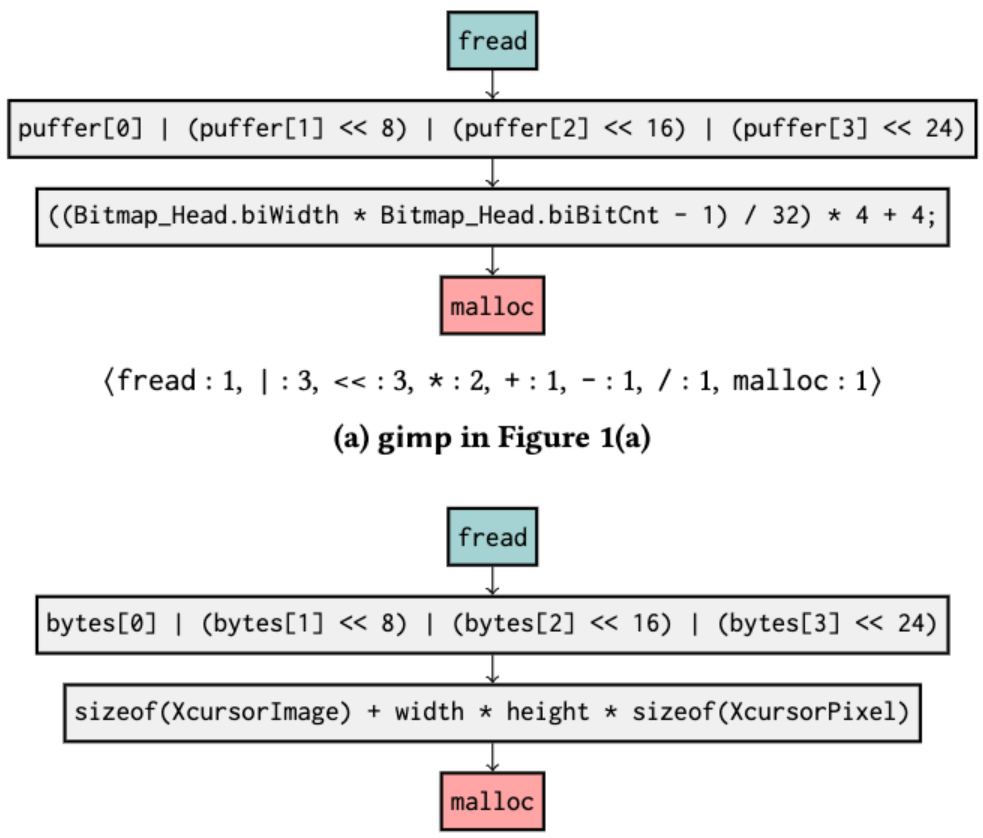
2.2.1 污点分析。Tracer 基于通用的污点分析，可以实例化为各种安全漏洞的检测器。该分析计算从不受信任的输入（源）到敏感函数（接收器）的潜在数据流，并使用一个简单的抽象域来表示受污染的值：$T = \{\bot, \top\}$，其中每个元素表示该值未受污染 ($\bot$) 和可能受污染 ($\top$)。例如，在图 1(a) 中，分析器检测到了从 fread 到 malloc 的恶意数据流。

为了进行更准确的分析，可以将其他抽象域与基本污染域结合使用。在我们的实现中，我们有一个简单的抽象域 $I = \{\bot, \top\}$，用于估计整数值是否可能溢出（$\top$）或不溢出（$\bot$）。例如，不受信任的输入值最初被污染（$\top$）但未溢出（$\bot$）。一旦该值用作可能导致整数溢出的运算符（例如，+，<<）的操作数，结果就会被污染（$\top$）和溢出（$\top$）。对于 malloc 情况，仅当参数的抽象值同时被污染（$\top$）和溢出（$\top$）时，我们的分析器才会发出警报。通过这样做，我们不会在有效计算恶意数据流的同时报告简单的误报。我们的实现细节在第 4 节中描述。

这里在讲如何检查警报吗？？？

2.2.2 数据依赖图上的跟踪。我们运行污点分析
对一组已知漏洞的程序进行分析。对于每个已知漏洞，Tracer 会根据静态分析结果，从源点和汇点提取漏洞踪迹。为了尽可能地过滤掉与漏洞无关的语句，我们会从数据中提取漏洞踪迹。

相当于做一个切片吧



(a) gimp in Figure 1(a)

(b) libXcursor in Figure 1(c)

Figure 3: Vulnerable traces and their feature vectors. The blue and red nodes represent the source and sink points, respectively.

dependency graphs rather than control-flow graphs. Once the taint analysis detects potentially malicious flows in gimp and libXcursor in Figure 1, Tracer derives data dependency graphs and extracts the vulnerable traces from the sources to sinks as shown in Figure 3. Such traces will be used as signatures of vulnerabilities.

The same procedure will be applied to new target programs. In particular, Tracer extracts all possible traces from sources to sinks of the reported alarms while unrolling each loop only once. These traces will be compared to the signature traces.

2.2.3 Feature Representation. Next, Tracer encodes each trace as an integer feature vector. We design a program-independent and common feature space that can represent transferable knowledge for vulnerabilities. Our feature vector consists of two parts: low-level and high-level features.

Low-level features represent the frequencies of primitive operators X (e.g., *, <<) and common APIs X (e.g., `strlen`) on the trace. Figure 3(a) shows the feature vector of the vulnerable trace in gimp. Likewise, the feature vector for libXcursor is shown in Figure 3(b).

On the other hand, high-level features describe detailed behavior of traces that are not noticeable using only the low-level ones. We manually designed 5 high-level features. In general, they characterize crucial behavior of programs that can affect our target vulnerabilities. For example, one of our features IfSmallerThanConst checks whether a trace has a conditional statement whose condition is of the form `x < c` where `x` is a variable and `c` is a constant. This pattern is common when programs prevent integer overflows. Suppose there exists such an expression in a trace of the target program, but not in the signature trace. Then, the target trace is deemed to be safe and the similarity score becomes lower.

Algorithm 1: Tracer$(\Pi, \mathcal{A}, P)$ where Π is a set of feature vectors of signature traces, $\mathcal{A}$ is a static analyzer, and P is the program to be analyzed.

1 $\Omega \leftarrow \mathcal{A}(P)$;
2 $G \leftarrow$ build_dfg(P);
3 $R \leftarrow \emptyset$;
4 **for** $\omega \in \Omega$ **do**
5 $\quad \mathcal{T}_\omega \leftarrow$ extract_traces(G, ω);
6 $\quad \Pi_\omega \leftarrow \{$generate_feature$(\tau) \mid \tau \in \mathcal{T}_\omega\}$;
7 $\quad s \leftarrow \max\{\text{Sim}(\pi_\omega, \pi) \mid \pi_\omega \in \Pi_\omega, \pi \in \Pi\}$;
8 $\quad R \leftarrow R\{\omega \mapsto s\}$;
9 **return** R;

$$\begin{array}{llll} \textit{Expression} & E & \rightarrow & n \mid x \mid E + E \mid E - E \mid \text{source}_l() \\ \textit{Command} & C & \rightarrow & x := E \mid \text{assume}(x < n) \mid \text{sink}(E) \end{array}$$

Figure 4: Language

2.2.4 Similarity Checking. Once a new program is analyzed, Tracer extracts all alarm traces and compares them against the known vulnerability signatures. Since all the traces are encoded as vectors, we can use any common similarity measures. In our implementation, we use cosine similarity, a well-known similarity measure for two vectors. For example, the cosine similarity of the two feature vectors in Figure 3 is computed as follows:

$$\frac{\langle 1,3,3,2,1,1,1,1\rangle \cdot \langle 1,3,3,2,1,0,0,1\rangle}{||\langle 1,3,3,2,1,1,1,1\rangle||\ ||\langle 1,3,3,2,1,0,0,1\rangle||} = 0.96$$

Therefore, Tracer can precisely detect semantically recurring vulnerabilities with high similarity scores.

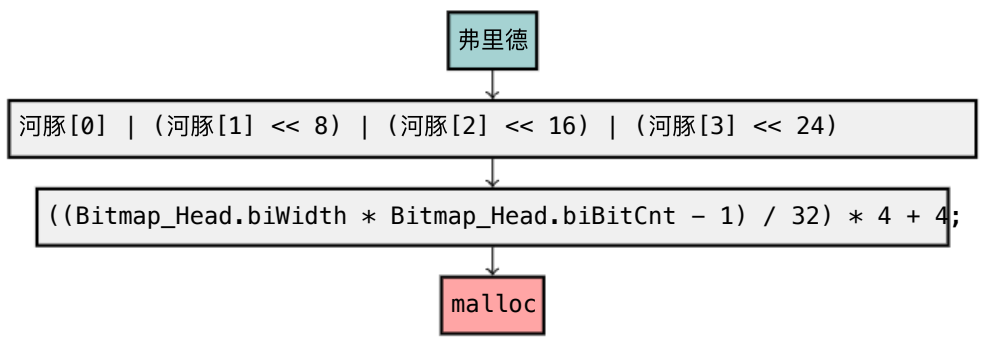
3 FRAMEWORK

In this section, we formalize our approach. The overall procedure of Tracer is described in Algorithm 1. Tracer first analyzes the target program and derives a set of alarms (line 1). Next, Tracer computes the data dependency graph of the program (line 2). For each alarm of the program, the algorithm extracts a set of traces (line 5) and encodes them as feature vectors (line 6). Finally, we compare each generated feature vector of the alarm ω with vulnerability signature traces. The score of the alarm is determined as the maximum similarity score of them (line 7). In the rest of this section, we formalize the details of each component of Tracer.

3.1 Program

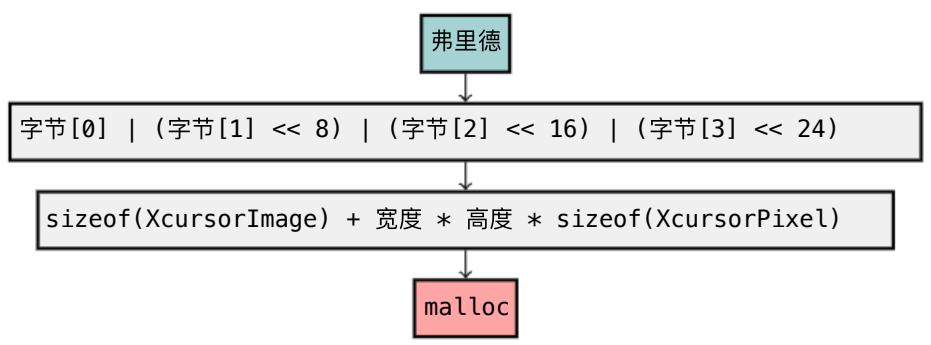
A program is represented as a control flow graph $\langle \mathbb{C}, \rightarrow \rangle$ where $\mathbb{C}$ is the set of control points and $(\rightarrow) \subseteq \mathbb{C} \times \mathbb{C}$ is the control-flow relation. Each control point is associated with a command. We assume a simple imperative language defined in Figure 4[1]. An expression is an integer, variable, addition operation, subtraction operation, or call to a source function. A command is an assignment, assume, or call to a sink function. `source` and `sink` represent functions

[1]For brevity, we only consider this simple language but our implementation handles the full features of C/C++.



⟨fread : 1, | : 3, << : 3, * : 2, + : 1, − : 1, / : 1, malloc : 1⟩

(a) 图 1（a）中的 gimp



⟨fread : 1, | : 3, << : 3, * : 2, + : 1, − : 0, / : 0, malloc : 1⟩

(b) 图 1（c）中的 libXcursor

图 3：脆弱轨迹及其特征向量。蓝色节点和红色节点分别代表源点和汇点。

依赖图而非控制流图。一旦污点分析检测到图 1 中 gimp 和 libXcursor 中的潜在恶意流程，Tracer 就会导出数据依赖图，并从源到接收器提取易受攻击的踪迹，如图 3 所示。

这些痕迹将被用作漏洞的签名。

相同的流程将应用于新的目标程序。具体来说，Tracer 会提取已报告警报从源到接收器的所有可能轨迹，同时仅展开每个循环一次。这些轨迹将与签名轨迹进行比较。

2.2.3 特征表示。接下来，Tracer 将每条跟踪信息编码为一个整数特征向量。我们设计了一个独立于程序且通用的特征空间，用于表示漏洞的可迁移知识。我们的特征向量由两部分组成：低级特征和高级特征。

低级特征表示原始运算符 X（例如 *、<<）和常用 API X（例如 strlen）在跟踪记录中出现的频率。图 3(a) 显示了 gimp 中存在漏洞的跟踪记录的特征向量。同样，libXcursor 的特征向量如图 3(b) 所示。

另一方面，高级特征描述了仅使用低级特征无法注意到的痕迹的详细行为。我们手动设计了 5 个高级特征。通常，它们表征了可能影响我们目标漏洞的程序的关键行为。例如，我们的特征之一 IfSmallerThanConst 检查痕迹是否具有条件语句，其条件的形式为 x < c，其中 x 是变量，c 是常量。这种模式在程序防止整数溢出时很常见。假设目标程序的痕迹中存在这样的表达式，但在签名痕迹中不存在。那么，目标痕迹被认为是安全的，相似度得分会降低。

算法 1：Tracer(Π, A, P)，其中 Π 是一组签名痕迹的特征向量，A 是静态分析器，P 是要分析的程序。

1 $\Omega \leftarrow$ A（P）；
2 $G \leftarrow$ build_dfg(P)；
3 $R \leftarrow \varnothing$；
4 对于$\omega \in \Omega$
5 　T←提取痕迹（G，ω）；
6 　Π← {生成特征(τ) | $\tau \in$ T}；
7 　$s \leftarrow$ max{Sim(π, π) | $\pi \varepsilon$ Π, $\pi \varepsilon$ Π}；
8 　$R \leftarrow R\{\omega \mapsto \rightarrow s\}$；
9 返回R；

$Expression\ E \rightarrow n \mid x \mid E + E \mid E - E \mid$ 源（） $Command\ C \rightarrow x := E \mid$ 假设（$x < n$）| 接收器（E）

图 4：语言

2.2.4 相似度检查。分析新程序后，Tracer 会提取所有警报跟踪，并将其与已知的漏洞签名进行比较。由于所有跟踪都被编码为向量，因此我们可以使用任何常见的相似度度量。在我们的实现中，我们使用余弦相似度，这是两个向量的常见相似度度量。例如，图 3 中两个特征向量的余弦相似度计算如下：

$$\frac{\langle 1,3,3,2,1,1,1,1\rangle \cdot \langle 1,3,3,2,1,0,0,1\rangle}{\|\langle 1,3,3,2,1,1,1,1\rangle\| \, \|\langle 1,3,3,2,1,0,0,1\rangle\|} = 0.96$$

因此，Tracer 可以精确检测具有高相似度得分的语义重复漏洞。

3 框架

在本节中，我们将形式化我们的方法。Tracer 的整体流程如算法 1 所示。Tracer 首先分析目标程序并生成一组警报（第 1 行）。接下来，Tracer 计算程序的数据依赖图（第 2 行）。对于程序的每个警报，该算法提取一组痕迹（第 5 行）并将其编码为特征向量（第 6 行）。最后，我们将警报ω的每个生成的特征向量与漏洞签名痕迹进行比较。警报的得分确定为它们的最大相似度得分（第 7 行）。在本节的其余部分，我们将形式化 Tracer 各个组件的细节。

3.1 程序

程序可以表示为控制流图 ⟨C, →⟩，其中 C 是控制点的集合，(→) ⊆ C×C 是控制流关系。每个控制点都与一个命令相关联。我们假设一种如图 4 所示的简单命令式语言。表达式可以是整数、变量、加法运算、减法运算或对源函数的调用。命令可以是赋值、假设或对接收函数的调用。源函数和接收函数分别表示函数

[1]为了简洁起见，我们只考虑这种简单的语言，但我们的实现处理了 C/C++ 的全部功能。

$$
\begin{array}{lrcl}
\text{(Abstract state)} & \mathbb{S} & = & \mathbb{C} \to \mathbb{M} \\
\text{(Abstract memory)} & \mathbb{M} & = & \mathbb{X} \to \mathbb{T} \times \mathbb{V} \\
\text{(Taint)} & \mathbb{T} & = & \wp(\mathbb{C})
\end{array}
$$

(a) Abstract domains

$$
\begin{array}{rcl}
[\![E]\!] & : & \mathbb{M} \to \mathbb{T} \times \mathbb{V} \\
[\![n]\!](m) & = & \langle \emptyset, \mathcal{V}(n)(m) \rangle \\
[\![x]\!](m) & = & m(x) \\
[\![E_1 + E_2]\!](m) & = & \langle T_1 \cup T_2, \mathcal{V}(E_1)(m) +_{\mathbb{V}} \mathcal{V}(E_2)(m) \rangle \\
[\![E_1 - E_2]\!](m) & = & \langle T_1 \cup T_2, \mathcal{V}(E_1)(m) -_{\mathbb{V}} \mathcal{V}(E_2)(m) \rangle \\
[\![\text{source}_l()]\!](m) & = & \langle \{l\}, \mathcal{V}(\text{source})(m) \rangle \\
\\
[\![C]\!] & : & \mathbb{M} \to \mathbb{M} \\
[\![x := E]\!](m) & = & m\{x \mapsto [\![E]\!](m)\} \\
[\![\text{assume}(x < n)]\!](m) & = & m \\
[\![\text{sink}(E)]\!](m) & = & m
\end{array}
$$

(b) Abstract semantics

Figure 5: Generic Taint Analysis

that read untrusted inputs (e.g., `fread`), and use the arguments in a sensitive context (e.g., `malloc`), respectively. We assume each source point is associated with a unique label l.

3.2 Generic Taint Analysis

We present a generic static analysis for taint tracking. The goal of the analysis is to estimate potential data flows from source points to sink points. The analysis can be instantiated to a family of taint analyses that are applicable to common types of vulnerabilities such as integer overflow, format string, or command injection [20, 21, 44]. We will present the detailed instantiation for our implementation in Section 4.

Abstract domains are shown in Figure 5(a). For a given program, our analyzer computes an abstract state ($\in \mathbb{S}$) that is a mapping from control points to the corresponding abstract memories. An abstract memory ($\in \mathbb{M}$) is a mapping from variables ($\in \mathbb{X}$) to their abstract values. An abstract value consists of two parts: the abstract domains for taint information ($\mathbb{T}$) and value information ($\mathbb{V}$). The taint domain is the power set of source labels. For taint checking, we collect all possible source points that lead to the value. The value domain represents general information of variables. For instance, one may define a simple abstract domain that only represents whether a value is overflowed, or a more sophisticated domain such as the interval domain. Our design choice will be explained in Section 4. Note that the value domain is not mandatory but is used to improve the precision of the analysis.

Abstract semantics is defined in Figure 5(b). The abstract semantics for expressions $[\![E]\!]$ computes the abstract value of an expression given an abstract memory. We assume that the value domain $\mathbb{V}$ is accompanied by an evaluation function $\mathcal{V} : E \to \mathbb{M} \to \mathbb{V}$ that computes the abstract value for an expression. Constant values (n) are not tainted and introduce an abstract value according to $\mathcal{V}$. For binary operations (+ and −), we join the taint information of two operands and compute the results of the corresponding abstract operator. For source points, the analyzer collects the labels, which will be used for taint checking, and computes its abstract value.

The abstract semantics for commands $[\![C]\!]$ computes the abstract memory after the execution of C given an abstract memory.

TRACER derives a set of alarms from the analysis results. An alarm of the taint analysis $\omega = \langle c_1, c_2 \rangle$ is a pair of two control points where c_1 is a source point and c_2 is a sink point that uses the untrusted data from the source c_1. We assume that the analysis is accompanied by an alarm inspection function $\mathcal{Q} : \mathbb{C} \to \mathbb{M} \to \wp(\mathbb{C})$. Given a sink point c and an abstract memory m at c from the analysis result, $\mathcal{Q}(c)(m)$ is a set of source points from which vulnerable data-flows start to the sink point c. Once an analysis $\mathcal{A}(P)$ for program P is completed, a set of alarms Ω is derived using the alarm inspection function.

DEFINITION 1 (ALARM). *Let $\mathbb{C}_s$ be a set of all sink points of a program. A set of alarms Ω of the program is defined as follows:*

$$\Omega = \{\langle c_1, c_2 \rangle \mid c_2 \in \mathbb{C}_s, c_1 \in \mathcal{Q}(c_2)(m)\}$$

where m is the abstract memory at c_2 according to the analysis results.

3.3 Data Dependency Graph and Tainted Traces

Next, we build a data dependency graph for the input program. Given a control-flow graph $\langle \mathbb{C}, \to \rangle$ of the program, the data dependency graph is defined as a tuple $\langle \mathbb{C}, \rightsquigarrow \rangle$. The data dependency graph has the same set of nodes but is based on data dependency relations rather than control-flow relations. We follow the standard notion of data dependency:

$$
\begin{aligned}
c_1 \rightsquigarrow c_2 \iff\ & c_1 \to^+ c_2 \ \wedge\ x \text{ is defined at } c_1 \ \wedge\ x \text{ is used at } c_2 \\
& \wedge\ x \text{ is not re-defined in any points between } c_1 \text{ and } c_2.
\end{aligned}
$$

where x is a program variable. Such data dependency relation can be computed during the static analysis by bookkeeping additional information about the definition and use points. While control dependencies are not considered, we capture crucial branch conditions by specially treating conditional expressions (`assume` in Figure 5(b)). We consider variables used in conditional expressions also as defined ones. With this heuristic choice, TRACER can capture important steps such as bound checking in practice.

Once a data dependency graph is established, we extract tainted traces of alarms. For each alarm, TRACER derives all paths from the source point to the sink point on the data dependency graph.

DEFINITION 2 (TAINTED TRACE). *Given an alarm $\omega = \langle c_0, c_n \rangle$, a set of tainted traces $\mathcal{T}_\omega \subseteq \mathbb{C}^+$ is defined as follows:*

$$\mathcal{T}_\omega = \{\langle c_0, \ldots, c_n \rangle \mid \forall i \in [0, n-1].\ c_i \rightsquigarrow c_{i+1}\}.$$

In the presence of loops, there can exist infinitely many traces of an alarm. In our implementation, TRACER unrolls each loop by once.

3.4 Feature Vector and Similarity Score

TRACER transforms each tainted trace to a feature vector that encodes the characteristics of the trace. We define a set of features to capture essential knowledge of vulnerable traces that are reusable across different programs. TRACER uses numerical features $f_i : \mathbb{C}^+ \to \mathbb{N}$. Given a set of n features $\{f_1, \ldots, f_n\}$, TRACER derives a feature vector of a trace τ: $\langle f_1(\tau), \ldots, f_n(\tau) \rangle$. Then, a set of feature vectors Π_ω of an alarm ω is defined as follows:

$$\Pi_\omega = \{\langle f_1(\tau), \ldots, f_n(\tau) \rangle \mid \tau \in \mathcal{T}_\omega\}$$

（抽象状态）S = C → M （抽象内存）M = X → T × V （污点）T = ℘(C)

（a）抽象域

[[E]] : M → T × V [[n]] (m) = ⟨∅， V (n)(m)⟩ [[x]] (m) = m(x)
[[E+ E]] (m) = ⟨T∪ T， V (E)(m) +V (E)(m)⟩ [[E− E]] (m) = ⟨T∪ T， V (E)(m) −V (E)(m)⟩ [[source()]] (m) = ⟨{l }， V （来源） (m) ⟩

[[C]] : M → M [[x := E]] (m) = m{x ↦ → [[E]] (m)} [[assume(x < n)]] (m) = m

[[sink(E)]] (m) = m

（b）抽象语义

图 5：通用污点分析

分别读取不受信任的输入（例如 fread）和在敏感上下文中使用参数（例如 malloc）。我们假设每个源点都与一个唯一的标签l相关联。

3.2 通用污点分析

我们提出了一种用于污点跟踪的通用静态分析方法。该分析的目标是估计从源点到汇点的潜在数据流。该分析可以实例化为一系列污点分析方法，适用于常见类型的漏洞，例如整数溢出、格式化字符串或命令注入[20, 21, 44]。我们将在第 4 节中详细介绍我们实现的具体实例。

抽象域如图 5(a) 所示。对于给定程序，我们的分析器计算一个抽象状态 (∈ S)，它是从控制点到相应抽象记忆的映射。抽象记忆 (∈ M) 是从变量 (∈ X) 到其抽象值的映射。抽象值由两部分组成：污点信息 (T) 和值信息 (V) 的抽象域。污点域是源标签的幂集。为了进行污点检查，我们收集所有可能导致该值的源点。值域表示变量的一般信息。例如，可以定义一个简单的抽象域，仅表示值是否溢出，或者定义一个更复杂的域，例如区间域。我们的设计选择将在第 4 节中解释。需要注意的是，值域不是强制性的，但它用于提高分析的精度。

抽象语义定义在图 5(b) 中。表达式 [[E]] 的抽象语义根据给定的抽象内存计算表达式的抽象值。我们假设值域 V 伴随一个求值函数 V : E → M → V，该函数计算表达式的抽象值。常量值 (n) 未受污染，并根据 V 引入抽象值。对于二元运算（+ 和 −），我们将两个操作数的污染信息连接起来，并计算相应抽象运算符的结果。对于源点，分析器收集将用于污染检查的标签，并计算其抽象值。

命令的抽象语义 [[C]] 在给定抽象记忆的情况下计算执行 C 后的抽象记忆。

追踪器从分析结果中得出一组警报。污点分析ω = ⟨c, c⟩的警报是一对两个控制点，其中c是源点，c是使用来自源c的不受信任数据的接收点。我们假设分析附带一个警报检查函数 Q : C → M → ℘(C)。给定一个接收点c和来自分析结果的c处的抽象内存m，Q (c)(m) 是一组源点，易受攻击的数据流从这些源点开始流向接收点c。一旦完成对程序P的分析 A (P)，就可以使用警报检查函数得出一组警报Ω。

定义 1 (警报)。设 C 是程序所有汇点的集合。该程序的一组警报 Ω 定义如下：

Ω = {⟨c, c⟩ | c∈C, c∈Q(c)(m)}

其中，m 是根据分析结果得出的 c 处的抽象记忆。

3.3 数据依赖图和污染痕迹

接下来，我们为输入程序构建数据依赖图。给定程序的控制流图 ⟨C, →⟩，数据依赖图定义为元组 ⟨C, ;⟩。数据依赖图具有相同的节点集，但基于数据依赖关系而非控制流关系。我们遵循数据依赖的标准概念：

这里本质就是做污点分析然后切片吧

c；c⇐⇒ c→c∧ x 在 c∧ 处定义 x 在 c 处使用
∧ x 在 c 和 c 之间的任何点上都没有重新定义。

其中x是程序变量。这种数据依赖关系可以在静态分析过程中通过记录定义点和使用点的附加信息来计算。虽然不考虑控制依赖关系，但我们通过特殊处理条件表达式来捕获关键的分支条件（如图 5(b) 所示）。我们将条件表达式中使用的变量也视为已定义的变量。通过这种启发式选择，Tracer 可以在实践中捕获诸如边界检查之类的重要步骤。

建立数据依赖图后，我们提取受污染的告警踪迹。对于每条告警，Tracer 会推导出数据依赖图上从源点到汇点的所有路径。

定义 2（污染踪迹）。给定一个警报 ω = ⟨c, c⟩，一组污染踪迹 T⊆ C 定义如下：

T = {⟨c， ...， c⟩ | ∀i ∈ [0， n − 1].c； c}。

如果存在循环，则警报的跟踪可能存在无限多个。在我们的实现中，Tracer 会将每个循环展开一次。

3.4 特征向量和相似度得分

Tracer 将每条受污染的痕迹转换为一个特征向量，该向量对痕迹的特征进行编码。我们定义了一组特征来捕获漏洞痕迹的必要知识，这些知识可在不同的程序之间重复使用。Tracer 使用数值特征 f: C→N。给定一组 n 特征 {f, . . . , f}，Tracer 会导出痕迹 τ 的特征向量：⟨f(τ), . . . , f(τ)⟩。然后，定义警报 ω 的一组特征向量 Π，如下所示：

Π= {⟨f(τ), . 。 。 , f(τ)⟩ | τ ∈ T}

$$
\begin{array}{rcl}
\text{(Abstract value)} & \mathbb{V} & = \bar{\mathbb{I}} \times \underline{\mathbb{I}} \\
\text{(Overflow)} & \bar{\mathbb{I}} & = \{\bot_o, \top_o\} \\
\text{(Underflow)} & \underline{\mathbb{I}} & = \{\bot_u, \top_u\} \\
\\
\mathcal{V}(n)(m) & = & \langle \bot_o, \bot_u \rangle \\
\mathcal{V}(E_1 + E_2)(m) & = & \langle \top_o, U_1 \sqcup U_2 \rangle \\
 & & \text{where } \mathcal{V}(E_1)(m) = \langle \_, U_1 \rangle \\
 & & \text{and } \mathcal{V}(E_2)(m) = \langle \_, U_2 \rangle \\
\mathcal{V}(E_1 - E_2)(m) & = & \langle O_1 \sqcup O_2, \top_u \rangle \\
 & & \text{where } \mathcal{V}(E_1)(m) = \langle O_1, \_ \rangle \\
 & & \text{and } \mathcal{V}(E_2)(m) = \langle O_2, \_ \rangle \\
\mathcal{V}(\text{source})(m) & = & \langle \bot_o, \bot_u \rangle
\end{array}
$$

Figure 6: Abstract domains

Finally, TRACER compares the feature vectors of alarms in program P to the set of all pre-computed feature vectors of known vulnerabilities, $\Pi_{\mathcal{S}}$. The set $\Pi_{\mathcal{S}}$ can be derived using the same steps described in the previous sections except that only known true alarms are considered. We also assume a function $\text{Sim} : \mathbb{N}^n \times \mathbb{N}^n \to \mathbb{R}$ that computes the similarity of two feature vectors. Using the similarity function, the score of an alarm is defined as the maximum similarity score between alarm traces and signature traces.

DEFINITION 3 (SCORE OF ALARM). *Given an alarm ω and a set of feature vectors of signatures $\Pi_{\mathcal{S}}$, the score of the alarm is defined as follows:*

$$\max\{Sim(\pi_\omega, \pi) \mid \pi_\omega \in \Pi_\omega, \pi \in \Pi_{\mathcal{S}}\}$$

where Π_ω is a set of feature vectors of alarm ω.

4 INSTANTIATION

This section describes the details of our system. First, we instantiate the generic taint analysis to detect common types of vulnerabilities. Our implementation aims at detecting integer overflows, integer underflows, buffer overflows, format string bugs, or command injections. Then, we explain our feature design.

4.1 Abstract Domains and Semantics

We define the abstract domain $\mathbb{V}$ and function $\mathcal{V}$ that are used in our implementation in Figure 6. The abstract domain $\mathbb{V}$ constitutes two parts: the overflow domain and the underflow domain. The overflow domain $\bar{\mathbb{I}}$ (resp., underflow domain $\underline{\mathbb{I}}$) represents whether the value may be overflowed ($\top_o$) (resp., underflowed) or not ($\bot_o$). The function $\mathcal{V} : E \to \mathbb{M} \to \mathbb{V}$ approximates the chances of integer overflows and underflows for a given expression and an abstract memory. Constant values (n) are not overflowed and underflowed. For addition (resp., subtraction) operators, we conservatively approximate the value to be potentially overflowed (resp., underflowed).

For the five types of vulnerabilities, we use the following alarm inspection function Q:

$$Q(c)(m) = Q_T(c)(m) \cup Q_O(c)(m) \cup Q_U(c)(m).$$

Each sub-function is defined as follows:

$$
\begin{aligned}
Q_T(c)(m) &= \{c_0 \mid c_0 \in T, \langle T, \_, \_ \rangle = [\![E]\!](m)\} \\
Q_O(c)(m) &= \{c_0 \mid c_0 \in T, \langle T, \top_o, \_ \rangle = [\![E]\!](m)\} \\
Q_U(c)(m) &= \{c_0, \mid c_0 \in T, \langle T, \_, \top_u \rangle = [\![E]\!](m)\}
\end{aligned}
$$

where c is a sink point and the abstract memory at c is m. Function Q_T collects all the source points of a sink point if the argument of a sink function is tainted. TRACER uses Q_T to detect format string, command injection, and buffer overflow at `printf`-like functions, `exec`-like functions, and `memcpy`-like functions, respectively. Q_O and Q_U additionally check whether the argument can be potentially overflowed and underflowed, respectively. The functions are used to detect malicious uses of memory allocations (e.g., `malloc`) with an overflowed (i.e., unintentionally small) argument, and memory copies (e.g., `memset`) with an underflowed (i.e., unintentionally large) argument.

4.2 Features and Similarity Measure

We have designed a set of features for tainted alarm traces that are shown in Table 1. The set of features comprises two categories: low-level and high-level features.

The low-level features NumOfOpX and NumOfLibX describe the frequency of each primitive operator X (e.g., + and <<) and standard library call X (e.g., `strlen` and `strcmp`) on a trace. For example, Figure 3 shows the feature vectors of traces with the low-level features. The motivation behind this feature design is based on the observation of typical bug reports. Developers typically recognize and describe a vulnerability in terms of a sequence of operations[2]. In our experience, such a sequence is a reasonable signature to characterize standard concepts such as geometry formulas or protocols. Even though our features only consider frequencies of operators on traces, each trace is carefully derived using a sophisticated context-sensitive static analysis. We extract traces only when the analyzer raises alarms. Furthermore, the traces are based on data flows rather than control flows. These design choices improve the accuracy of the system based on the similarity comparison.

On the other hand, the high-level features are designed to capture deeper contexts of traces. Instead of counting individual occurrences of operators, the features describe relationships among expressions and operators. We separated the low-level features from the high-level ones to balance manual efforts and accuracy. The low-level features consist of general program components that are transferable across different programs and do not require manual efforts. Instead, we manually designed the high-level features by observing typical bug-fix patterns. For example, EqualToPercentage is inspired by usual conditional expressions to prevent format string bugs. A similar design choice is used in MVP [47]; they also handle format strings specially. Such high-level features help filter out typical false alarms. In our experiments, the accuracy of TRACER with only the low-level features is already reasonably high, but the high-level features can further improve the accuracy. The details will be discussed in Section 5.4.

TRACER uses cosine similarity, a well-known measure of similarity between two vectors. Given two feature vectors π_1 and π_2, the

[2]For example, https://cgit.freedesktop.org/xorg/lib/libXcursor/commit/?id=4794b5dd34688158fb51a2943032569d3780c4b8.

（抽象值） V = I × I（溢出） I = {⊥, ⊤}（下溢）

我={⊥，⊤}

V (n)(m) = ⟨⊥, ⊥⟩
V (E+ E)(m) = ⟨⊤, U⊔ U⟩
其中 V (E)(m) = ⟨_, U⟩ 和 V (E)(m) = ⟨_, U⟩ V (E− E)(m) = ⟨O⊔ O, ⊤⟩
其中 V (E)(m) = ⟨O, _⟩ 和 V (E)(m) = ⟨O, _⟩
V (来源)(m) = ⟨⊥, ⊥⟩

图 6：抽象域

最后，Tracer 将程序 P 中警报的特征向量与已知漏洞的所有预先计算的特征向量集合 Π 进行比较。集合 Π 可以使用上一节中描述的相同步骤导出，不同之处在于只考虑已知的真实警报。我们还假设一个函数 Sim：N× N→R，用于计算两个特征向量的相似度。使用相似度函数，将警报的得分定义为警报轨迹与签名轨迹之间的最大相似度得分。

定义 3（警报评分）。给定一个警报ω和一组特征向量Π，报警得分定义如下：

最大{Sim(π, π) | πε Π, π ε Π}

其中 π 是警报 ω 的一组特征向量。

4 实例化

本节将介绍我们系统的细节。首先，我们将实例化通用污点分析，以检测常见类型的漏洞。我们的实现旨在检测整数溢出、整数下溢、缓冲区溢出、格式字符串错误或命令注入。然后，我们将解释我们的功能设计。

4.1 抽象域和语义

我们定义了图 6 中实现中使用的抽象域 V 和函数 V。抽象域 V 包含两部分：溢出域和下溢域。溢出域 I（或下溢域 I）表示值是否可能溢出（或下溢）或不会溢出（或下溢）。函数 V：$E \to$ M → V 近似计算给定表达式和抽象内存发生整数溢出和下溢的概率。常量值 (n) 不会溢出或下溢。对于加法（或减法）运算符，我们保守地近似计算可能溢出（或下溢）的值。

针对这五类漏洞，我们使用如下的报警检查函数 Q：

Q (c)(m) = Q(c)(m) ∪ Q(c)(m) ∪ Q(c)(m)。

各个子功能定义如下：

Q(c)(m) = {c| c∈T, ⟨T, _, _⟩ = [[E]] (m)} Q(c)(m) = {c| c∈T, ⟨T, ⊤, _⟩ = [[E]] (m)} Q(c)(m) = {c, | c∈T, ⟨T, _, ⊤⟩ = [[E]] (m)}

其中c是接收点，c处的抽象内存是m。如果接收函数的参数被污染，函数 Q 会收集接收点的所有源点。Tracer 使用 Q 分别检测类似 printf 函数、类似 exec 函数和类似 memcpy 函数的格式字符串、命令注入和缓冲区溢出。Q 和 Q 还会分别检查参数是否可能溢出和下溢。这两个函数用于检测带有溢出（即无意中较小）参数的内存分配（例如 malloc）的恶意使用，以及带有下溢（即无意中较大）参数的内存复制（例如 memset）。

4.2 特征和相似性度量

我们针对污染报警痕迹设计了一组特征，如表 1 所示。该组特征包括两类：低级特征和高级特征。

低级特征 NumOfOpX 和 NumOfLibX 描述了跟踪上每个原始运算符 X（例如，+ 和 <<）和标准库调用 X（例如，strlen 和 strcmp）的频率。例如，图 3 显示了具有低级特征的跟踪的特征向量。此功能设计背后的动机基于对典型错误报告的观察。开发人员通常根据操作序列来识别和描述漏洞。根据我们的经验，这样的序列是表征标准概念（例如几何公式或协议）的合理特征。即使我们的功能仅考虑跟踪上运算符的频率，每个跟踪都是使用复杂的上下文敏感静态分析精心得出的。我们仅在分析器发出警报时才提取跟踪。此外，跟踪基于数据流而不是控制流。这些设计选择提高了基于相似性比较的系统准确性。

另一方面，高级特征旨在捕捉跟踪的更深层次上下文。这些特征不是计算运算符的单个出现次数，而是描述表达式和运算符之间的关系。我们将低级特征与高级特征分开，以平衡人工工作量和准确性。低级特征由通用程序组件组成，可在不同程序之间迁移，无需人工操作。相反，我们通过观察典型的错误修复模式手动设计了高级特征。例如，EqualToPercentage 的灵感来自通常的条件表达式，用于防止格式字符串错误。MVP [47] 中使用了类似的设计选择；它们也特殊处理格式字符串。这些高级特征有助于滤除典型的误报。在我们的实验中，仅使用低级特征的 Tracer 准确率已经相当高，但高级特征可以进一步提高准确率。详细信息将在 5.4 节中讨论。

Tracer 使用余弦相似度，这是两个向量之间一个众所周知的相似度度量。给定两个特征向量π和π，

[8] 例如，https://cgit.freedesktop.org/xorg/lib/libXcursor/commit/?id=4794b5dd34688158fb51a2943032569d3780c4b8。

Table 1: Features of traces. E and K represent an arbitrary expression and a constant, respectively.

Name	Description
NumOfOpX	# primitive operator X on the trace
NumOfLibX	# calls to library X on the trace
LargerThanConst	# expressions of the form $E > K$ or $E \geq K$
SmallerThanConst	# expressions of the form $E < K$ or $E \leq K$
EqualToVar	# expressions of the form $E == K$
NotEqualToVar	# expressions of the form E != K
EqualToPercentage	# expressions of the form E == '%'

similarity is defined as follows:

$$\text{Sim}(\pi_1, \pi_2) = \frac{\pi_1 \cdot \pi_2}{||\pi_1||\ ||\pi_2||}.$$

4.3 Application to Other Vulnerability Types

The general principle behind TRACER—computing similarity scores between traces—is applicable to other types of vulnerabilities if the underlying analyzer produces traces of alarms. For example, static analyzers for double-free or use-after-free bugs typically report potentially erroneous traces from a call to `free` to other calls to `free` or uses of the same pointer. We demonstrate the applicability to other vulnerability types in the next section.

5 EXPERIMENT

Our evaluation is designed to answer the following questions:

- **RQ1**: How effective is TRACER for finding unknown recurring vulnerabilities?
- **RQ2**: How accurate is TRACER compared with existing approaches?
- **RQ3**: How effective is the high-level features of TRACER?
- **RQ4**: How scalable is TRACER to large programs?

All experiments were conducted on Linux machines with Intel Xeon 2.90GHz. We set the timeout to one hour for running the static analysis for each package. Our code and data are publicly available at https://prosys.kaist.ac.kr/tracer/.

5.1 Experimental Setup

5.1.1 Implementation. We implemented TRACER on top of Facebook's Infer analyzer [5]. The taint analyzer is designed as described in the previous sections. We use pointer information computed by Infer's buffer overrun checker. Following Infer's framework, our taint analysis is designed to be a modular interprocedural analysis (i.e., context-sensitive). For each benchmark, we run 20 tasks in parallel. Our taint analysis checks five common vulnerabilities described in Section 4: integer overflows, integer underflows, buffer overflows, command injections, and format string bugs. We used the Pulse engine of Infer for use-after-free and double-free bugs.

5.1.2 Signature programs. We collected signature programs from different sources of real-world and synthetic vulnerabilities:

(1) **Real-world vulnerabilities**: We collected 16 vulnerabilities that can be reproduced by our taint analysis from the CVE report [10] and prior work [20, 21].
(2) **Juliet test suite** [4]: Juliet Test Suite consists of a large set of small programs each of which has a common vulnerability. We used 5,383 programs that have the same types of vulnerabilities handled by our analysis.
(3) **Online tutorial**: We collected 5 examples from online tutorials on secure programming provided by OWASP [16].

5.1.3 Benchmarks. We evaluated TRACER using 273 Debian packages written in C/C++. We selected 16 common categories (web, sound, utils, etc) of Debian packages [13] that contain at least one package that Infer can analyze. This step excludes many categories consisting of unsupported languages (e.g., R, Haskell, etc) and packages having build issues in our environment. We randomly chose 20 packages that have at least one alarm raised by our analyzer in the selected categories. For categories that have less than 20 packages, we used all packages in the categories.

5.1.4 Baselines. We compare TRACER with state-of-the-art bug detection tools from three categories: *1)* clone-based approach *2)* learning-based approach *3)* pattern-based static analyzer. For each category, we chose tools that were recently proposed and are publicly available: VUDDY [26], CCAligner [43], Devign [48] and Github's CodeQL [2]. We ran the baselines for the same types of vulnerabilities handled by TRACER. For VUDDY, we selected the reported alarms based on their CWE ID [11] that matches the vulnerability types. For CodeQL, we ran all their security-related queries dedicated to the corresponding CWE IDs [7] of the types.

5.1.5 Metrics. Each analyzer reports alarms in different manners. For example, tools based on static analysis (e.g., CodeQL, Infer) typically report sink points as alarms. However, this may be an overestimation if there are multiple sink points from the same malicious input point. For a fair comparison, we used the following metrics:

- **Root causes**: To show the effectiveness (Section 5.2), we report the number of root causes of discovered vulnerabilities. We manually inspected the true alarms from the analyzers and counted the number of root causes.
- **Sink points**: To compare the precision and recall of the analyzers (Section 5.3), we use sink points as both TRACER and CodeQL provide them to the users as alarms. We counted the number of true alarms and false alarms reported by the analyzers. For the other tools (VUDDY, CCAligner, Devign) that report vulnerable functions, we counted the number of reported functions.

5.2 RQ1: Effectiveness

5.2.1 New Bugs. This section shows how effective TRACER is for detecting previously unknown vulnerabilities in the Debian packages. We manually inspected all the reported alarms whose similarity scores are larger than 0.85. In addition, we randomly selected 100 alarms below the threshold and manually inspected them. In total, 424 reports (i.e., sink points) were investigated.

TRACER found 112 new vulnerabilities in 67 packages. Among them, 30 vulnerabilities have been confirmed by the developers and 6 CVEs have been assigned as of writing this paper. For all the other reports, we have not received answers by the time of submission. Among the 112 bugs, only 10 can be found by the baseline tools (VUDDY_O and Devign_O).

表 1：迹的特征。E 和 K 分别表示任意表达式和常数。

姓名	描述
NumOfOpX	# 轨迹上的原始运算符 X
NumOfLibX	# 在跟踪中对库 X 的调用
LargerThanConst	# 形式为 $E > K$ 或 $E \geqslant K$ 的表达式
SmallerThanConst	# 形式为 $E < K$ 或 $E \leqslant K$ 的表达式
EqualToVar	# 形式为 $E == K$ 的表达式
NotEqualToVar	# 形式为 E != K 的表达式
EqualToPercentage	# 形式为 E == '%' 的表达式

相似度定义如下：

$$\text{尝试}(\pi, \pi) = \frac{\pi \cdot \pi}{\|\pi\| \, \|\pi\|}.$$

4.3 应用于其他漏洞类型

Tracer 背后的通用原理——计算轨迹之间的相似度得分——如果底层分析器生成了警报轨迹，则也适用于其他类型的漏洞。例如，针对双重释放或释放后使用漏洞的静态分析器通常会报告从一次释放调用到其他释放调用或使用同一指针的潜在错误轨迹。下一节我们将演示其对其他漏洞类型的适用性。

5 实验

我们的评估旨在回答以下问题：

- RQ1：Tracer 对于查找未知的重复漏洞有多有效?
- RQ2：与现有方法相比，Tracer 的准确度如何?
- RQ3：Tracer 的高级功能效果如何?
- RQ4：Tracer 对大型程序的可扩展性如何?

所有实验均在搭载 Intel Xeon 2.90GHz 处理器的 Linux 机器上进行。我们将每个软件包的静态分析运行超时时间设置为 1 小时。我们的代码和数据已在 https://prosys.kaist.ac.kr/tracer/ 公开发布。

5.1 实验设置

5.1.1 实现。我们在 Facebook 的 Infer 分析器 [5] 之上实现了 Tracer。污点分析器的设计如前几节所述。我们使用由 Infer 的缓冲区溢出检查器计算出的指针信息。遵循 Infer 的框架，我们的污点分析被设计为模块化的过程间分析（即上下文敏感）。对于每个基准测试，我们并行运行 20 个任务。我们的污点分析检查第 4 节中描述的五种常见漏洞：整数溢出、整数下溢、缓冲区溢出、命令注入和格式字符串错误。我们使用 Infer 的 Pulse 引擎来检测释放后使用 (UAF) 和双重释放 (Double-Free) 错误。

5.1.2 签名程序。我们从不同的真实和合成漏洞来源收集了签名程序：

(1) 现实世界的漏洞：我们从 CVE 报告 [10] 和之前的工作 [20, 21] 中收集了 16 个可以通过污点分析重现的漏洞。

(2) Juliet 测试套件 [4]：Juliet 测试套件由大量小程序组成，每个程序都存在一个常见的漏洞。我们使用了 5,383 个程序，这些程序的漏洞类型与我们的分析结果相同。

(3) 在线教程:我们从 OWASP [16] 提供的安全编程在线教程中收集了 5 个示例。

5.1.3 基准测试。我们使用 273 个用 C/C++ 编写的 Debian 软件包对 Tracer 进行了评估。我们选择了 16 个常见的 Debian 软件包类别（Web、声音、实用程序等），这些类别至少包含一个 Infer 可以分析的软件包。此步骤排除了许多包含不受支持的语言（例如 R、Haskell 等）以及在我们的环境中存在构建问题的软件包的类别。我们随机选择了 20 个在所选类别中至少被我们的分析器触发过一次警报的软件包。对于软件包数量少于 20 个的类别，我们使用了这些类别中的所有软件包。

5.1.4 基线。我们将 Tracer 与以下三个类别的最先进的漏洞检测工具进行比较：1）基于克隆的方法 2）基于学习的方法 3）基于模式的静态分析器。对于每个类别，我们都选择了最近提出且可公开获取的工具：VUDDY [26]、CCAligner [43]、Devign [48] 和 Github 的 CodeQL [2]。我们针对 Tracer 处理的相同类型漏洞运行了基线。对于 VUDDY，我们根据与漏洞类型匹配的 CWE ID [11] 选择已报告的警报。对于 CodeQL，我们针对相应的 CWE ID [7] 运行了所有与安全相关的查询。

5.1.5 指标。每个分析器报告警报的方式各不相同。例如，基于静态分析的工具（例如 CodeQL、Infer）通常将接收点报告为警报。但是，如果同一恶意输入点有多个接收点，则这可能会被高估。为了公平起见，我们使用了以下指标：

- 根本原因：为了证明有效性（见第 5.2 节），我们报告了已发现漏洞的根本原因数量。我们手动检查了分析器发出的真实警报，并统计了根本原因的数量。
- 汇点：为了比较分析器的准确率和召回率（见第 5.3 节），我们使用汇点作为警报，因为 Tracer 和 CodeQL 都将其作为警报提供给用户。我们统计了分析器报告的真实警报和错误警报的数量。对于其他报告漏洞函数的工具（VUDDY、CCAligner、Devign），我们统计了报告的函数数量。

5.2 RQ1：有效性

5.2.1 新漏洞。本节展示了 Tracer 在检测 Debian 软件包中未知漏洞方面的有效性。我们手动检查了所有相似度得分大于 0.85 的已报告警报。此外，我们还随机选择了 100 个低于阈值的警报并进行了手动检查。总共调查了 424 份报告（即汇聚点）。

Tracer 在 67 个软件包中发现了 112 个新漏洞。其中 30 个漏洞已得到开发人员确认，截至本文撰写时已分配 6 个 CVE。对于所有其他报告，截至提交时我们尚未收到答复。在这 112 个漏洞中，只有 10 个可以通过基准工具（VUDDY 和 Devign）发现。

```
bool ssgLoadTGA(...) {
    GLubyte header[18];
    fread(header, 18, 1, f);
    ...
    int xsize = get16u(header + 12);
    int ysize = get16u(header + 14);
    int bits = header[16];
    ...
    // potential integer overflow
    GLubyte *image = new GLubyte [ (bits / 8) * xsize * ysize ];
    ...
}

int get16u(const GLubyte *ptr) { return (ptr[0] | (ptr[1] << 8)); }
```

Figure 7: An integer overflow discovered in libplib1-1.8.5 that is similar to the one from sam2p-0.49.4 in Figure 1(b).

```
void CWE190_Integer_Overflow__int64_t_fscanf_square_01_bad() {
    int64_t data;
    data = 0LL;
    fscanf (stdin, "%" SCNd64, &data);
    // potential integer overflow
    int64_t result = data * data;
    char *p = malloc(result);
}
```

(a) Juliet test suite (CWE-190)

```
static DiaObject *fig_read_polyline(FILE *file, DiaContext *ctx) {
    fscanf(file, "%d %d %d %d %d %d %d %d %d %lf %d %d\n", ..., &npoints)
    newobj = create_standard_polyline(npoints, ...);
    ...
}

DiaObject *create_standard_polyline(int num_points, ...) {
    pcd.num_points = num_points;
    new_obj = otype->ops->create(NULL, &pcd, &h1, &h2);
    ...
}

static DiaObject *polyline_create(Point *startpoint, void *user_data,
    Handle **handle1, Handle **handle2) {
    MultipointCreateData *pcd = (MultipointCreateData *)user_data;
    polyconn_init(poly, pcd->num_points);
    ...
}

void polyconn_init(PolyConn *poly, int num_points) {
    // potential integer overflow
    poly->points = g_malloc(num_points * sizeof(Point));
    ...
}
```

(b) dia-0.97.3

Figure 8: An integer overflow bug in dia-0.97.3 and a signature vulnerability from Juliet test suite.

Table 2 shows the vulnerabilities discovered by Tracer. We observed that Tracer can detect various types of vulnerabilities using signatures from different sources including known CVEs or synthetic vulnerabilities. Most of the detected vulnerabilities have high similarity scores. We will discuss the detailed distribution of the scores in the next section.

Tracer is able to detect new vulnerabilities that are similar to known ones. Figure 7 shows a vulnerability found in libplib1. The signature that gives the highest score for the case is sam2p in Figure 1(b) which is itself a recurring vulnerability similar to

```
int main(void) {
    char *ptr_h;
    char h[64];
    ptr_h = getenv("HOME");
    if (ptr_h != NULL) {
        // potential buffer overflow
        sprintf(h, "Your home directory is: %s !", ptr_h);
        printf("%s\n", h);
    }
    return 0;
}
```

(a) OWASP tutorial

```
XrmDatabase resource_buildDatabase(...) {
    char locale1[100];
    char loc_lang[100];
    char *locale = getenv("LC_ALL");
    String s = getenv("XUSERFILESEARCHPATH");
    char *cP = loc_lang;
    char *cL = locale;
    ...
    while (*cL) {
        ...
        *cP++ = *cL++;
    }
    *cP = 0;

    if (s == NULL || !strcasecmp(s, "False")) {
        // potential buffer overflow
        sprintf(locale1, "noint:%s%s", loc_lang, ...);
        ...
    }
}
```

(b) gv-3.7.4

Figure 9: A buffer overflow bug in gv-3.7.4 and a signature vulnerability from an OWASP tutorial.

```
generic *gp_alloc(size_t size, ...) { /* wrapper of malloc */ }

static int LUA_init_lua(void) {
    char *script_fqn;
    char *gp_lua_dir = getenv("GNUPLOT_LUA_DIR");
    ...
    // allocation with a large enough length
    script_fqn = gp_alloc(strlen(gp_lua_dir) + ... + 2, ...);
    // potential buffer overflow (false alarm)
    sprintf(script_fqn, "%s%c%s", gp_lua_dir, ...);
    ...
}
```

(a) gnuplot-5.2.8

```
SHPHandle SHPAPI_CALL SHPOpenLL(...) {
    SHPHandle psSHP;
    uchar *pabyBuf = (uchar *)malloc(100);
    fread(pabyBuf, 100, 1, psSHP->fpSHX);
    psSHP->nRecords = pabyBuf[27] + pabyBuf[26] * 256 + ...;
    ...
    // bound checking
    if (psSHP->nRecords > 256000000) {
        return (NULL);
    }
    ...
    // false alarm (integer overflow)
    int32 *panSHX = (int32 *)malloc(sizeof(int32) * 2 * psSHP->nRecords);
}
```

(b) grass-7.8.2

Figure 10: False alarms filtered by the similarity measure of Tracer

```
1 bool ssgLoadTGA(...) {
2 GLubyte 头[18];
3 fread(header, 18, 1, f);
4 ...
5 int xsize = get16u(header + 12);
6 int ysize = get16u(header + 14);
7 int 位 = 标头[16];
8 ...
9 // 潜在的整数溢出
10 GLubyte *image = new GLubyte [ (位 / 8) * xsize * ysize ];
11 ...
12 }
13
14 int get16u(const GLubyte *ptr) { return (ptr[0] | (ptr[1] << 8)); }
```

图 7：在 libplib1-1.8.5 中发现的整数溢出

与图 1(b) 中 sam2p-0.49.4 的类似。

```
1 无效 CWE190_Integer_Overflow__int64_t_fscanf_square_01_bad () {
2 int64_t 数据;
3 数据=0LL;
4 fscanf (stdin, "%" SCNd64, &data);
5 // 潜在的整数溢出
6 int64_t 结果 = 数据 * 数据;
7 char *p = malloc(结果);
8 }
```

（a）Juliet 测试套件（CWE-190）

```
1 静态 DiaObject *fig_read_polyline(FILE *file, DiaContext *ctx) {
2 fscanf（文件，“%d␣%d␣%d␣%d␣%d␣%d␣%d␣%d␣%lf␣%d␣%d \ n”，...，&npoints）
3 newobj = 创建_标准_折线（npoints，...）；
4 ...
5 }
6
7 DiaObject *create_standard_polyline(int num_points, ...) {
8 pcd.num_points = num_points;
9 new_obj = otype->ops->create(NULL, &pcd, &h1, &h2);
10 ...
11 }
12
13 静态 DiaObject *polyline_create(Point *startpoint, void *user_data,
14 句柄**handle1，句柄**handle2）{
15 多点创建数据*pcd =（多点创建数据*)用户数据;
16 polyconn_init (poly, pcd->num_points）；
17 ...
18 }
19
20 void polyconn_init(PolyConn *poly, int num_points) {
21 // 潜在的整数溢出
22 poly->points = g_malloc(num_points * sizeof(Point));
23 ...
24 }
```

（b）直径-0.97.3

图 8：dia-0.97.3 中的整数溢出漏洞和 Juliet 测试套件中的签名漏洞。

表 2 列出了 Tracer 发现的漏洞。我们观察到，Tracer 可以使用来自不同来源（包括已知 CVE 和合成漏洞）的签名来检测各种类型的漏洞。大多数检测到的漏洞都具有较高的相似度得分。我们将在下一节讨论这些得分的详细分布情况。

Tracer 能够检测到与已知漏洞类似的新漏洞。图 7 展示了 libplib1 中发现的一个漏洞。该案例得分最高的签名是图 1(b) 中的 sam2p，它本身也是一个重复出现的漏洞，类似于

```
1 int main (void) {
2 字符*ptr_h;
3 字符 h[64];
4 ptr_h = getenv(“家”);
5 如果 (ptr_h! = NULL) {
6 // 潜在的缓冲区溢出
7 sprintf(h, "您的主目录是: %s! ", ptr_h);
8 printf("%s\n", h);
9 }
10 返回 0;
11 }
```

（一）OWASP 教程

```
1 XrmDatabase resource_buildDatabase(...) {
2 char locale1[100];
3 字符 loc_lang[100];
4 char *locale = getenv("LC_ALL");
5 字符串 s = getenv("XUSERFILESEARCHPATH");
6 字符 *cP = loc_lang;
7 char *cL = 区域设置;
8 ...
9 当 (*cL) {
10 ...
11 *cp++=*cl++;
12 }
13 *cp=0;
14
15 如果 (s == NULL || ! strcasecmp (s, “False”) ) {
16 // 潜在的缓冲区溢出
17 sprintf(locale1, "noint:%s%s", loc_lang, ...);
18 ...
19 }
20 }
```

（b）gv-3.7.4

图 9：gv-3.7.4 中的缓冲区溢出错误和来自 OWASP 教程的签名漏洞。

```
1 通用 *gp_alloc(size_t size, ...) { /* malloc 的包装器 */}
2
3 静态 int LUA_init_lua(void) {
4 字符*script_fqn;
5 char *gp_lua_dir = getenv("GNUPLOT_LUA_DIR");
6 ...
7 // 分配足够长的长度
8 script_fqn = gp_alloc(strlen(gp_lua_dir) + ... + 2, ...);
9 // 潜在的缓冲区溢出（误报）
10 sprintf(script_fqn, "%s%c%s", gp_lua_dir, ...);
11 ...
12 }
```

（一）gnuplot-5.2.8

```
1 SHPHandle SHPAPI_CALL SHPOpenLL(...) {
2 SHPHandle psSHP;
3 浮点*pabyBuf = (浮点*)malloc(100);
4 fread(pabyBuf, 100, 1, psSHP->fpSHX);
5 psSHP->nRecords = pabyBuf[27] + pabyBuf[26] * 256 + ...;
6 ...
7 // 边界检查
8 如果（psSHP->nRecords> 256000000）{
9 返回（NULL）；
10 }
11 ...
12 // 误报（整数溢出）
13 int32 *panSHX = (int32 *)malloc(sizeof(int32) * 2 * psSHP->nRecords);
14 }
```

（b）草-7.8.2

图 10：通过 Tracer 的相似性度量过滤的误报

Table 2: List of new vulnerabilities detected by TRACER. #Bugs reports the number of bugs by root causes, Signature shows the sources of vulnerability signatures and Score represents the similarity scores between the bugs and the signatures. ✔ indicates the vulnerabilities whose CVE IDs have been assigned. Type indicates the type of bugs (IO: integer overflow, IU: integer underflow, BO: buffer overflows, CI: command injection, FO: format string bug, UAF: use-after-free, DF: double-free).

Program	#Bugs	Type	Score	Signature	CVE
4ti2	1	IO	0.71-1.00	Juliet-CWE190	-
abyss	1	IO	0.82	Juliet-CWE190	-
alsa-utils	1	DF	1	Juliet-CWE415	-
bowtie2	1	IO	0.74	CVE-2017-9181	-
bsdutils	1	CI	0.86	CVE-2016-10729	-
bsdutils	1	IO	1	Juliet-CWE190	✔
bwbasic	1	BO	0.44	CVE-2018-1100	-
coinor-libcgl1	1	IO	0.87	Juliet-CWE190	-
coinor-libclp1	1	IO	1	Juliet-CWE190	-
crafty	1	IO	0.86	CVE-2017-1000229	-
cron	1	CI	0.68	OWASP tutorial	-
crrcsim	2	IO	0.85-0.90	CVE-2017-16663	-
darktable	3	IO	1	Juliet-CWE680	-
dcraw	1	IO	0.93-0.94	CVE-2017-9181	✔
dia	1	IO	1	Juliet-CWE190	✔
drawxtl	1	IO	0.79	CVE-2017-9181	-
dvbstreamer	1	BO	1	OWASP tutorial	-
elvis-tiny	2	BO	0.50-1.00	OWASP tutorial	-
gap-guava	1	IO	1	Juliet-CWE190	-
gnuplot	2	FS	0.82	Juliet-CWE134	-
grass	8	BO	0.41-1.00	OWASP tutorial	-
groff	1	IO	1	Juliet-CWE190	-
gv	1	BO	1	OWASP tutorial	-
htmldoc	2	IO	0.90-0.95	CVE-2017-9181	✔
hugin	2	IO	0.87-1.00	Juliet-CWE190	-
ispell	2	BO	1	OWASP tutorial	-
libaudio2	1	BO	1	OWASP tutorial	-
libfreeimage3	1	BO	0.83	CVE-2017-6313	-
libkrb5support0	1	IO	1	Juliet-CWE190	-
liblinear-tools	1	BO	0.3	CVE-2018-1100	-
liblinear-tools	1	IO	0.93-1.00	Juliet-CWE190	-
liblrs0	1	IO	0.91	Juliet-CWE191	-
libmjpegutils-2.1-0	1	BO	1	OWASP tutorial	-
libmount1	1	CI	0.72	CVE-2015-9059	-
libmount1	1	IO	1	Juliet-CWE190	-
libpano13-3	1	FS	0.59	mp3rename-0.6 [20]	✔
libpano13-3	1	IO	0.87	CVE-2017-16663	-
libplib1	5	IO	0.76-0.93	shntool-3.0.5 [20]	✔
libquicktime2	1	IO	0.85-0.95	CVE-2017-9181	-

Program	Bugs	Type	Score	Signature	CVE
lp-solve	1	IO	1	Juliet-CWE190	-
mailutils	1	UAF	1	Juliet-CWE415	-
mdadm	1	BO	0.16	CVE-2019-14523	-
minidlna	1	IO	0.94	Juliet-CWE190	-
nageru	1	IO	0.87	CVE-2017-16663	-
nedit	1	BO	1	OWASP tutorial	-
newmail	1	FS	0.82	Juliet-CWE134	-
nickle	1	BO	1	OWASP tutorial	-
nickle	1	CI	0.67	Juliet-CWE78	-
octave-nan	3	IO	0.87-1.00	Juliet-CWE190	-
printer-driver-foo2zjs	1	IU	0.94	Juliet-CWE191	-
r-cran-lpsolve	1	IO	1	Juliet-CWE190	-
rawtherapee	4	IO	0.86-1.00	Juliet-CWE680	-
rlwrap	1	CI	0.82	Juliet-CWE78	-
rtcw	1	BO	0.4	CVE-2018-1100	-
sa-exim	1	CI	1	Juliet-CWE78	
sane	1	IO	0.87	CVE-2017-9181	-
scheme48	1	IO	0.85	CVE-2009-1570	-
seaview	1	BO	0.56	CVE-2018-1100	-
siril	2	IO	0.87-1.00	Juliet-CWE680	-
siril	1	IU	0.82	Juliet-CWE191	-
snap	2	BO	1	OWASP tutorial	-
snap	1	IO	1	Juliet-CWE680	-
stk	1	IO	0.87	shntool-3.0.5 [20]	-
sweed	1	IO	1	Juliet-CWE190	-
tcliis	1	BO	1	OWASP tutorial	-
tome	1	FS	0.96	CVE-2015-8106	-
vacation	1	CI	0.67	Juliet-CWE78	-
w3m	1	FS	0.96	CVE-2015-8106	-
wily	5	BO	0.47-1.00	OWASP tutorial	-
xbuffy	1	BO	1	OWASP tutorial	-
xfig	2	BO	1	OWASP tutorial	-
xsane	1	IO	0.87-1.00	Juliet-CWE190	-
xwpe	1	BO	1	OWASP tutorial	-
xwpe	1	CI	0.87	Juliet-CWE78	-
xwpe	3	IO	0.87-0.89	Juliet-CWE190	-
zangband	1	BO	0.77	CVE-2017-6313	-
zangband	1	FS	0.97	CVE-2015-8106	-
zangband	1	IO	0.93	CVE-2017-9181	-

the one in Figure 1(a). We also notice that TRACER can effectively discover real-world security bugs using synthetically generated toy examples. Figure 8 shows an integer overflow vulnerability in dia detected by TRACER and a signature vulnerability from the Juliet test suite. Notice that they have completely different syntactic structures. For example, the vulnerability in dia involves three function calls including one indirect call, as well as complicated pointer dereferences. On the other hand, synthetic code has an extremely simple structure. However, they have the same root cause of the vulnerabilities. Both of the programs read an external input using `fscanf`, and cause an integer overflow by multiplying the input value with another integer value. TRACER exactly detects the same vulnerable behavior from the two programs and sets the similarity score to 1.0.

For other types of vulnerabilities, TRACER can also detect recurring vulnerabilities that are similar to existing ones. Figure 9 depicts a buffer overflow error in gv. This bug happens because the program reads an untrusted string using `getenv` that is used to construct a new string via `sprintf`. The vulnerable behavior is described in a tutorial by OWASP [36]. Likewise, TRACER detects one use-after-free and one double-free bug similar to examples in the Juliet test suite. While the example codes are simple, the real-world vulnerability involves complicated aliases, indirect assignments, and control flows. Nevertheless, TRACER can find that the essence

表 2：Tracer 检测到的新漏洞列表。#Bugs 报告按根本原因划分的漏洞数量，Signature 表示漏洞签名的来源，Score 表示漏洞与签名之间的相似度得分。" 表示已分配 CVE ID 的漏洞。Type 表示漏洞的类型（IO：整数溢出，IU：整数下溢，BO：缓冲区溢出，CI：命令注入，FO：格式字符串漏洞，UAF：释放后使用，DF：双重释放）。

程序	#Bugs	类型	分数	签名
4ti2	1	IO	0.71–1.00	朱丽叶–CWE190 –
深渊	1	IO	0.82	Juliet–CWE190 –
alsa–utils	1	DF	1	Juliet–CWE415 –
bowtie2	1	IO	0.74	CVE–2017–9181 –
bsdutils	1	CI	0.86	CVE–2016–10729 –
bsdutils	1	IT	1	朱丽叶–CWE190 "
bwbasic	1	BO	0.44	CVE–2018–1100 –
coinor–libcgl1	1	IO	0.87	朱丽叶–CWE190 –
coinor–libclp1	1	IO	1	Juliet–CWE190 –
crafty	1	IO	0.86	CVE–2017–1000229 –
cron	1	CI	0.68	OWASP 教程 –
crrcsim	2	IO	0.85–0.90	CVE–2017–16663 –
darktable	3	IO	1	Juliet–CWE680 –
直流电流	1	IO	0.93–0.94	CVE–2017–9181 "
旅行	1	IT	1	朱丽叶–CWE190 "
drawxtl	1	IO	0.79	CVE–2017–9181 –
dvbstreamer	1	BO	1	OWASP 教程 –
elvis–tiny	2	BO	0.50–1.00	OWASP 教程 –
gap–guava	1	IO	1	Juliet–CWE190 –
gnuplot	2	FS	0.82	Juliet–CWE134 –
草	8	BO	0.41–1.00	OWASP 教程 –
groff	1	IO	1	Juliet–CWE190 –
gv	1	BO	1	OWASP 教程 –
htmldoc	2	IO	0.90–0.95	CVE–2017–9181 "
hugin	2	IO	0.87–1.00	Juliet–CWE190 –
ispell	2	BO	1	OWASP 教程 –
libaudio2	1	BO	1	OWASP 教程 –
libfreeimage3	1	BO	0.83	CVE–2017–6313 –
libkrb5support0	1	IO	1	Juliet–CWE190 –
liblinear–tools	1	BO	0.3	CVE–2018–1100 –
liblinear–tools	1	IO	0.93–1.00	Juliet–CWE190 –
liblrs0	1	IO	0.91	Juliet–CWE191 –
libmjpegutils–2.1–0	1	BO	1	OWASP 教程
libmount1	1	CI	0.72	CVE–2015–9059 –
libmount1	1	IO	1	Juliet–CWE190 –
libpano13–3	1	FS	0.59	mp3rename–0.6 [20]
libpano13–3	1	IO	0.87	CVE–2017–16663 –
libplib1	5	IO	0.76–0.93	shntool–3.0.5 [20]
libquicktime2	1	IO	0.85–0.95	CVE–2017–9181 –
lp–解决	1	IO	1	Juliet–CWE190 –
mailutils	1	UAF	1	朱丽叶–CWE415 –
mdadm	1	BO	0.16	CVE–2019–14523 –
minidlna	1	IO	0.94	Juliet–CWE190 –
nageru	1	IO	0.87	CVE–2017–16663 –
nedit	1	BO	1	OWASP 教程 –
新邮件	1	FS	0.82	Juliet–CWE134 –
nickle	1	BO	1	OWASP 教程 –
镍	1	CI	0.67	Juliet–CWE78 –
octave–nan	3	IO	0.87–1.00	Juliet–CWE190 –
打印机驱动程序–foo2zjs	1	IU	0.94	Juliet–CWE191
r–cran–lpsolve	1	IO	1	Juliet–CWE190 –
rawtherapee	4	IO	0.86–1.00	Juliet–CWE680 –
rlwrap	1	CI	0.82	朱丽叶–CWE78 –
rtcw	1	BO	0.4	CVE–2018–1100 –
sa–exim	1	CI	1	Juliet–CWE78 –
健康	1	IO	0.87	CVE–2017–9181 –
scheme48	1	IO	0.85	CVE–2009–1570 –
海景	1	BO	0.56	CVE–2018–1100 –
siril	2	IO	0.87–1.00	Juliet–CWE680 –
siril	1	IU	0.82	Juliet–CWE191 –
snap	2	BO	1	OWASP 教程 –
快照	1	IO	1	Juliet–CWE680 –
stk	1	IO	0.87	shntool–3.0.5 [20] –
瑞典	1	IO	1	朱丽叶–CWE190 –
tcliis	1	BO	1	OWASP 教程 –
tome	1	FS	0.96	CVE–2015–8106 –
假期	1	CI	0.67	Juliet–CWE78 –
w3m	1	FS	0.96	CVE–2015–8106 –
wily	5	BO	0.47–1.00	OWASP 教程 –
xbuffv	1	BO	1	OWASP 教程 –
xfig	2	BO	1	OWASP 教程 –
xsane	1	IO	0.87–1.00	Juliet–CWE190 –
xwpe	1	BO	1	OWASP 教程 –
xwpe	1	从	0.87	Juliet–CWE78 –
xwpe	3	IO	0.87–0.89	Juliet–CWE190 –
zangband	1	BO	0.77	CVE–2017–6313 –
zangband	1	FS	0.97	CVE–2015–8106 –
赞班德	1	IT	0.93	CVE–2017–9181 –

图 1(a) 中的代码。我们还注意到，Tracer 可以使用合成生成的玩具示例有效地发现现实世界的安全漏洞。图 8 展示了 Tracer 检测到的 dia 中的整数溢出漏洞和 Juliet 测试套件中的签名漏洞。请注意，它们具有完全不同的语法结构。例如，dia 中的漏洞涉及三个函数调用（包括一个间接调用）以及复杂的指针取消引用。另一方面，合成代码的结构非常简单。然而，它们具有相同的漏洞根源。这两个程序都使用 fscanf 读取外部输入，并通过将输入值与另一个整数值相乘来导致整数溢出。Tracer 准确地检测到

从两个程序中找出相同的脆弱行为，并将相似度得分设为 1.0。

对于其他类型的漏洞，Tracer 还可以检测到与现有漏洞类似的重复出现漏洞。图 9 展示了 gv 中的一个缓冲区溢出错误。发生此错误的原因是程序使用 getenv 读取了一个不受信任的字符串，而该字符串又通过 sprintf 构造了一个新字符串。OWASP 的教程 [36] 中描述了此漏洞行为。同样，Tracer 检测到一个与 Juliet 测试套件中的示例类似的 UAF 漏洞和一个双重释放漏洞。虽然示例代码很简单，但实际漏洞涉及复杂的别名、间接赋值和控制流。尽管如此，Tracer 还是可以发现漏洞的本质

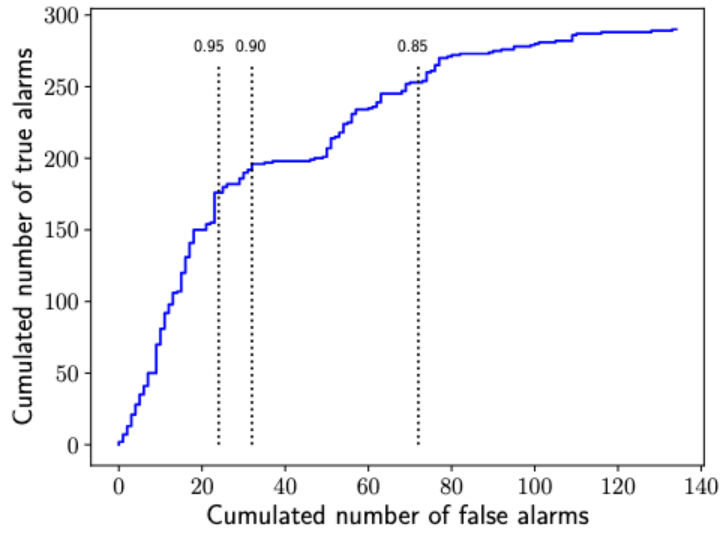


Figure 11: Ranking effectiveness. Each data point represents the number of true and false alarms that the user will obtain when enumerating the sorted alarms by ranking.

of the bug is actually the same as the tutorial examples as the static analyzer estimates the detailed semantics.

5.2.2 Ranking Effectiveness. Next, we evaluate the effectiveness of our ranking scheme. For all the manually inspected 424 alarms, we investigate where true and false positives are located in the bug reports ranked by their similarity scores.

Figure 11 and the top three rows in Table 3 show that most of the true positives are highly ranked in the report. While the underlying static analysis reports 1,975 alarms, only 176 of them have scores higher than 0.95 (TRACER$_{95}$). Among them, 154 (87.5%) are confirmed as true positives. The true positive ratio is still high (85.7% and 78.1%) if we set the threshold to 0.9 (TRACER$_{90}$) and 0.85 (TRACER$_{85}$), respectively. The similarity-based score of TRACER effectively filters out a large number of false positives while retaining many real bugs. These results indicate that TRACER can help developers effectively discover recurring vulnerabilities.

TRACER's similarity measure not only prioritizes similar bugs, but also effectively suppresses false alarms. Figure 10(a) shows an alarm that is falsely identified by our taint analysis in gnuplot. This alarm looks similar to the vulnerability in Figure 9(a). However, the call to `sprintf` is safe because the program always allocates enough memory blocks using `strlen` and multiple addition operations. These operators are captured by our features and differentiate this alarm from typical vulnerable patterns. Figure 10(b) shows another example of a false alarm in grass. The integer overflow will never occur since there is a bound checking for external user inputs. This is also captured by one of the high-level features LargerThanConst. Notice that these features only appear in the false alarm traces, not in the signatures. This in turn leads to a lower score for these alarms (0.77–0.88) and ranks them below many other true alarms.

Overall, the experimental results show that TRACER is effective to detect semantically recurring vulnerabilities. In particular, our trace-based similarity measure powered by static analysis is robust to syntactic variants. Thus, TRACER can report recurring vulnerabilities with high similarity scores even though two programs have significantly different syntactic characteristics.

Table 3: Comparison to state-of-the-art approaches. Column RC, TP, FP and FN report the number of vulnerabilities by root causes, true positives, false positives and false negatives reported by each analyzer, respectively.

Analyzer	RC	TP	FP	FN	Prec (%)	Recall (%)
TRACER$_{95}$	58	154	22	299	87.5	34.0
TRACER$_{90}$	69	192	32	261	85.7	42.4
TRACER$_{85}$	87	253	71	200	78.1	55.8
VUDDY$_O$	3	5	7	448	41.7	1.1
VUDDY$_S$	0	0	10	453	0.0	0.0
CCAligner	0	0	150	453	0.0	0.0
CodeQL	86	161	163	292	49.7	35.5
Devign$_O$	-	10	-	443	-	2.2
Devign$_S$	-	0	-	453	-	0.0

5.3 RQ2: Comparison

This section compares the accuracy of TRACER with the state-of-the-art tools. In the rest of this section, we use TRACER$_{85}$ for comparison because all its reports are manually inspected. Since there is no fully labeled data set for recurring vulnerabilities and it is hard to manually inspect all vulnerabilities in our benchmarks, we set up a set of ground truths as follows:

- We ran all the tools (TRACER$_{85}$, VUDDY, CCAligner, and CodeQL) and manually inspected the same number of alarms as TRACER$_{85}$ (i.e., 324). If a tool reports fewer than 324 alarms, we inspected all of them.
- We collected the true alarms detected by all the tools and included the true alarms from the random sampling in Section 5.2, which are not detected by TRACER$_{85}$.

In total, we collected 453 ground truths and compared the accuracy of TRACER$_{85}$ to the other baselines.

Table 3 shows the performance of each analyzer. Note that we could not use Devign [48], a learning-based approach, to collect ground truths. Unlike the other tools, Devign does not provide explainable reports, such as vulnerabilities they are similar to (e.g., TRACER, VUDDY, CCAligner) or description of vulnerabilities (e.g., CodeQL). Thus, we only investigated whether Devign can detect any of 453 ground truths.

5.3.1 Comparison to VUDDY and CCAligner. We compared TRACER against clone-based detectors, VUDDY and CCAligner. For VUDDY, we established two different settings in ways to collect the vulnerability database for clone detection. VUDDY$_O$ is based on the original database provided by the official web service that has 1,764 CVEs as signatures [22]. In order to discard the effect of the quality of the database per se, we also tried VUDDY$_S$ that uses our own signature database. Following the same methodology as VUDDY$_O$, we collected all the vulnerable functions that are patched in the later versions for the real-world benchmarks, or annotated in the source code for the synthetic benchmarks. For CCAligner, we followed the same setting as VUDDY$_S$. The threshold of CCAligner is set to 0.85 which is the same setting as used in TRACER.

VUDDY$_O$ reports 7 false positives out of 12 alarms. The reason for the false alarms is due to a practical issue regarding establishing their databases. VUDDY$_O$ collects all the modified functions in

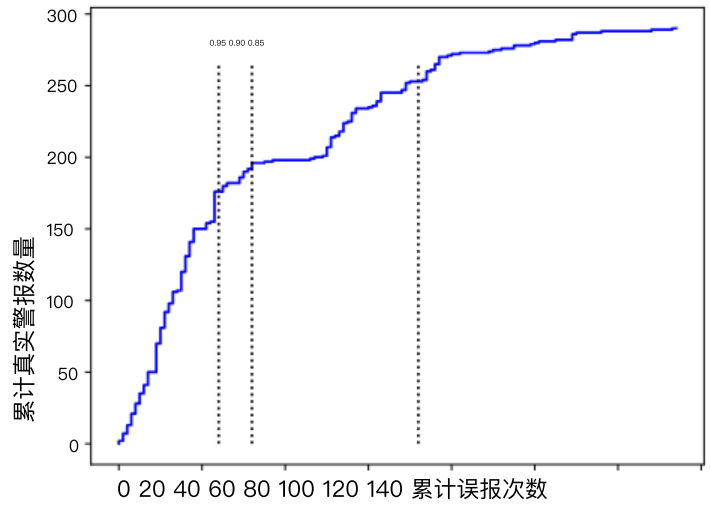


图 11：排序有效性。每个数据点代表用户按排序枚举已排序警报时将获得的真实警报和错误警报的数量。

该错误的本质与教程示例相同，因为静态分析器会估计详细的语义。

5.2.2 排序有效性。接下来，我们评估排序方案的有效性。对于所有手动检查的 424 条警报，我们调查了按相似度得分排序的错误报告中真阳性和假阳性的位置。

图 11 和表 3 的前三行显示，大多数真阳性在报告中排名较高。底层静态分析报告了 1,975 条告警，但其中只有 176 条的评分高于 0.95（Tracer）。其中，154 条（87.5%）被确认为真阳性。即使我们将阈值分别设置为 0.9（Tracer）和 0.85（Tracer），真阳性率仍然很高（85.7% 和 78.1%）。Tracer 基于相似度的评分有效地过滤了大量的假阳性，同时保留了大量真正的漏洞。这些结果表明，Tracer 可以帮助开发人员有效地发现重复出现的漏洞。

Tracer 的相似性度量不仅可以优先处理相似的漏洞，还能有效抑制误报。图 10(a) 显示了我们在 gnuplot 中进行污点分析时错误识别的警报。此警报与图 9(a) 中的漏洞类似。但是，对 sprintf 的调用是安全的，因为程序始终使用 strlen 和多个加法运算分配足够的内存块。这些运算符被我们的特征捕获，并将此警报与典型的漏洞模式区分开来。图 10(b) 显示了 Grass 中另一个误报示例。由于对外部用户输入进行了边界检查，因此永远不会发生整数溢出。高级特征 LargerThanConst 也捕获了这一点。请注意，这些特征仅出现在误报轨迹中，而不出现在签名中。这反过来导致这些警报的得分较低（0.77–0.88），并使其排名低于许多其他真实警报。

总体而言，实验结果表明，Tracer 能够有效检测语义上重复出现的漏洞。尤其值得一提的是，我们基于静态分析的轨迹相似度测量方法对句法变体具有鲁棒性。因此，即使两个程序的句法特征存在显著差异，Tracer 也能报告相似度得分较高的重复出现漏洞。

表 3：与最新方法的比较。RC、TP、FP 和 FN 列分别报告了每个分析器报告的漏洞数量（按根本原因划分）、真正例、误报和漏报。

分析器	RC	TP	FP	FN	准确率 (%)	召回率 (%)
Tracer58	154	22	299	87.5	34.0	
Tracer69	192	32	261	85.7	42.4	
Tracer87	253	71	200	78.1	55.8	
VUDDY3	5	7	448	41.7	1.1	
VUDDY0	0	10	453	0.0	0.0	
CCAligner	0	0	150	453	0.0	0.0
CodeQL	86	161	163	292	49.7	35.5
Devign–	10	–	443	–	2.2	
Devign	–	0	–	453	–	0.0

5.3 RQ2：比较

本节将 Tracer 与最新工具的准确率进行比较。在本节的其余部分，我们将使用 Tracer 进行比较，因为其所有报告均经过手动检查。由于目前没有针对重复出现漏洞的完整标记数据集，并且很难在我们的基准测试中手动检查所有漏洞，因此我们设置了一组基本事实，如下所示：

- 我们运行了所有工具（Tracer、VUDDY、CCAligner 和 CodeQL），并手动检查了与 Tracer 相同数量的警报（即 324 条）。如果某个工具报告的警报少于 324 条，则我们会检查所有警报。
- 我们收集了所有工具检测到的真实警报，并包括了第 5.2 节中随机抽样的真实警报，这些警报未被 Tracer 检测到。

总共，我们收集了 453 个基本事实，并将 Tracer 的准确性与其他基线进行了比较。

表 3 展示了各分析器的性能。需要注意的是，我们无法使用 Devign [48]（一种基于学习的方法）来收集基本事实。与其他工具不同，Devign 不提供可解释的报告，例如与其相似的漏洞（例如 Tracer、VUDDY、CCAligner）或漏洞描述（例如 CodeQL）。因此，我们仅考察了 Devign 能否检测到 453 个基本事实中的任何一个。

5.3.1 与 VUDDY 和 CCAligner 的比较。我们比较了 Tracer 针对基于克隆的检测器 VUDDY 和 CCAligner。对于 VUDDY，我们建立了两种不同的设置来收集用于克隆检测的漏洞数据库。VUDDY 基于官方网络服务提供的原始数据库，该数据库包含 1,764 个 CVE 作为签名 [22]。为了消除数据库本身质量的影响，我们还尝试了使用我们自己的签名数据库的 VUDDY。遵循与 VUDDY 相同的方法，我们收集了所有在后续版本中为真实基准测试修补的漏洞函数，或在合成基准测试的源代码中注释的漏洞函数。对于 CCAligner，我们遵循与 VUDDY 相同的设置。CCAligner 的阈值设置为 0.85，与 Tracer 中使用的设置相同。

VUDDY 报告了 12 次警报中的 7 次误报。误报的原因是由于建立数据库时出现了实际问题。VUDDY 收集了所有修改后的函数

```
1  unsigned sget4 (unsigned char *s) {
2      ...
3      return s[0] << 24 | s[1] << 16 | s[2] << 8 | s[3];
4  }
5
6  unsigned get4() {
7      unsigned char str[4] = { 0xff,0xff,0xff,0xff };
8      fread (str, 1, 4, ifp);
9      return sget4((unsigned char *)str);
10 }
11
12 void foveon_load_camf() {
13     unsigned wide = get4();
14     unsigned high = get4();
15     ...
16     // potential integer overflow
17     char *meta_data = (char *) malloc(wide * high * 3/2);
18     ...
19 }
```

Figure 12: A vulnerability found in dcraw-9.28 and rawtherapee-5.8.

```
void badVaSink(char *data, ...) {
    va_list args;
    va_start(args, data);
    vfprintf(stdout, data, args);
    va_end(args);
}
```

(a) Juliet test suite (CWE-134)

```
void lqt_dump(char * format, ...) {
    va_list argp;
    va_start(argp, format);
    vfprintf(stdout, format, argp);
    va_end(argp);
}
```

(b) libquicktime2-1.2.4

Figure 13: A code clone detected by VUDDY$_S$

patch commits of known CVEs as signatures. However, a single commit may contain numerous irrelevant modifications. This leads to spurious signatures that match non-vulnerable functions. In fact, all of the false positives from VUDDY$_O$ turned out to be the case.

VUDDY$_O$ detects 3 vulnerabilities by reporting 5 function clones. Among them, TRACER can detect two vulnerabilities but misses one because there is no similar trace in our signature database. The commonly detected vulnerabilities are found in dcraw and rawtherapee, both being exactly the same functions as shown in Figure 12. The function `foveon_load_camf` reads `wide` and `high` from an external file (line 13–14), and allocates memory after multiplication (line 17) that can cause a potential integer overflow. VUDDY$_O$ reports that this bug originated from the same vulnerable source (LibRaw-demosaic-pack-GPL2, CVE-2017-6889). TRACER detected these vulnerabilities with a high similarity score (0.92) even though the origin is not in our signature database. Instead, TRACER captures that the vulnerability is similar to the one in sam2p shown in Figure 1(b). Notice that the bug from sam2p has a totally different syntactic structure from the code in Figure 12. This example demonstrates that TRACER effectively generalizes known vulnerability patterns to detect unseen ones.

Even though the database is carefully established with only vulnerable functions, VUDDY$_S$ reports 10 false alarms and no true bugs. The behavior of a function often depends on the context in which it is used. For instance, the function in Figure 13(a) is a signature in our database. If the argument `data`, which is passed to the second argument of `vfprintf`, can be controlled by an attacker, this in turn causes format string vulnerability. With this signature,

```
1  int32_t endianSwapI32(int32_t i) {
2      int32_t tmp = 0;
3      tmp |= ((i >> 24) & (0xff << 0));
4      tmp |= ((i >> 8) & (0xff << 8));
5      tmp |= ((i << 8) & (0xff << 16));
6      tmp |= ((i << 24) & (0xff << 24));
7      return tmp;
8  }
9
10 T readI(FILE *in, ...) {
11     T x;
12     fread((void *)&x, 1, sizeof(T), in);
13     ...
14     if (sizeof(T) == 4) {
15         return endianSwapI32(x);
16     }
17     ...
18 }
19
20 void Ebwt::readIntoMemory(...) {
21     uint32_t _nPat;
22     FILE *_in1;
23     string _in1Str;
24     ...
25     _in1 = fopen(_in1Str.c_str(), "rb"));
26     _nPat = readI<uint32_t>(_in1, ...);
27     // false alarm (integer overflow)
28     _plen.init(new uint32_t[_nPat], _nPat, true);
29 }
```

Figure 14: A false positive reported by CodeQL recognized as a potential integer overflow bug in bowtie2-2.3.5.1.

VUDDY$_S$ detects function `lqt_dump` in Figure 13(b) as a recurring vulnerability. However, according to our manual investigation, all the calls to `lqt_dump` takes only safe format arguments. Therefore this function is not vulnerable in the context of libquicktime2.

CCAligner reports 150 function clones but does not detect any vulnerability. Since CCAligner is not designed to find vulnerabilities, functions that do not have security-sensitive calls are reported. Similar to VUDDY, CCAligner also detects function clones without considering their contexts as in Figure 13(b). These lead to a large number of false positives in vulnerability detection.

Both VUDDY and CCAligner cannot detect most of the recurring vulnerabilities detected by TRACER. This is mainly because they are based on syntactic similarity measures at the function-level granularity. However, most of the vulnerabilities detected by TRACER involve multiple functions and have significantly different syntactic structures from signatures. Such characteristics in real-world programs hinder those tools from detecting semantically recurring vulnerabilities.

5.3.2 Comparison to CodeQL. In this section, we compare TRACER to CodeQL, which is a static analysis with human-written bug patterns (i.e., queries). We applied CodeQL with the queries related to the same types of vulnerabilities as TRACER, that are available in their repository[3]. In total, CodeQL reports 3,488 alarms from the benchmark programs. Among them we inspected 324 alarms.

Our experiments show that TRACER effectively detects recurring vulnerabilities that are missed by CodeQL. CodeQL reports 161 true alarms (35.5%) out of 453 ground truths while TRACER$_{85}$ detects 253

[3]https://github.com/github/codeql/blob/main/cpp/ql/src/Security/CWE/

```
1 无符号 sget4 (无符号 char *s) {
2 ...
3 返回 s[0] << 24 | s[1] << 16 | s[2] << 8 | s[3];
4 }
5
6 无符号 get4() {
7 无符号字符 str[4] = { 0xff,0xff,0xff,0xff };
8 fread (str, 1, 4, ifp) ;
9 返回 sget4((unsigned char *)str);
10 }
11
12 无效 foveon_load_camf () {
13 无符号宽 = get4();
14 无符号高 = get4();
15 ...
16 // 潜在的整数溢出
17 char *meta_data = (char *) malloc(宽*高*3/2);
18 ...
19 } 图 12：dcraw-9.28 和 rawtherapee-5.8 中发现的漏洞。
```

```
void badVaSink(char *data, ...) {
va_list args; va_start(args, data);
vfprintf(stdout, data, args);
va_end(args); }
```

```
void lqt_dump(char * format, ...) {
va_list argp; va_start(argp, 格式);
vfprintf (标准输出, 格式, argp) ;
va_end(argp); }
```

（a）Juliet 测试套件（CWE-134） （b）libquicktime2-1.2.4

图 13：VUDDY 检测到的代码克隆

已知 CVE 的补丁提交会被用作签名。然而，单个提交可能包含大量不相关的修改。这会导致出现与非漏洞函数匹配的伪签名。事实上，VUDDY 的所有误报都是这种情况。

VUDDY 通过报告 5 个函数克隆检测到 3 个漏洞。其中，Tracer 检测到两个漏洞，但漏掉了一个，因为我们的签名数据库中没有类似的漏洞踪迹。常见的漏洞出现在 dcraw 和 rawtherapee 中，这两个函数完全相同，如图 12 所示。foveon_load_camf 函数从外部文件读取宽和高（第 13–14 行），并在乘法运算后分配内存（第 17 行），这可能导致潜在的整数溢出。VUDDY 报告称，此漏洞源自同一漏洞源（LibRaw-demosaic-pack-GPL2，CVE-2017-6889）。Tracer 检测到这些漏洞的相似度得分很高（0.92），尽管漏洞源不在我们的签名数据库中。相反，Tracer 捕获到该漏洞与图 1(b) 中所示的 sam2p 中的漏洞类似。请注意，sam2p 中的错误具有与图 12 中的代码完全不同的语法结构。此示例表明 Tracer 可以有效地概括已知的漏洞模式以检测看不见的漏洞。

尽管数据库是精心构建的，只包含易受攻击的函数，但 VUDDY 报告了 10 个误报，没有真正的错误。函数的行为通常取决于其使用环境。例如，图 13(a) 中的函数是我们数据库中的一个签名。如果传递给 vfprintf 函数第二个参数的参数数据可以被攻击者控制，这反过来会导致格式字符串漏洞。使用此签名，

```
1 int32_t endianSwapI32 (int32_t i) {
2 int32_t tmp = 0;
3 tmp |= ((i >> 24) & (0xff << 0));
4 tmp |= ((i >> 8) & (0xff << 8));
5 tmp |= ((i << 8) & (0xff << 16));
6 tmp |= ((i << 24) & (0xff << 24));
7 返回 tmp;
8 }
9
10 T 读取 (FILE *in, ...) {
11 T x;
12 fread((void *)&x, 1, sizeof(T), in);
13 ...
14 如果 (sizeof (T) == 4) {
15 返回 endianSwapI32(x);
16 }
17 ...
18 }
19
20 void Ebwt::readIntoMemory(...) {
21 uint32_t _nPat;
22 文件 *_in1;
23 字符串 _in1Str;
24 ...
25 _in1 = fopen(_in1Str.c_str(), "rb"));
26 _nPat = 读取 I (_in1, ...) ;
27 // 误报 (整数溢出)
28 _plen.init(new uint32_t[_nPat], _nPat, true);
29 } 图 14：CodeQL 报告的误报被识别为 bowtie2-2.3.5.1 中的潜在整数溢出错误。
```

VUDDY 检测到图 13(b) 中的 lqt_dump 函数存在重复漏洞。然而，根据我们的手动调查，所有对 lqt_dump 的调用都只接受安全格式的参数。因此，该函数在 libquicktime2 环境中不存在漏洞。

CCAligner 报告了 150 个函数克隆，但未检测到任何漏洞。由于 CCAligner 并非旨在查找漏洞，因此会报告不包含安全敏感调用的函数。与 VUDDY 类似，CCAligner 也会在不考虑其上下文的情况下检测函数克隆，如图 13(b) 所示。这会导致漏洞检测中出现大量误报。

VUDDY 和 CCAligner 都无法检测到 Tracer 检测到的大多数重复性漏洞。这主要是因为它们基于函数级粒度的语法相似性度量。然而，Tracer 检测到的大多数漏洞涉及多个函数，并且其语法结构与签名存在显著差异。实际程序中的这些特性阻碍了这些工具检测语义重复性的漏洞。

5.3.2 与 CodeQL 的比较。在本节中，我们将 Tracer 与 CodeQL 进行比较，后者是一种使用人工编写的漏洞模式（即查询）进行静态分析的工具。我们将 CodeQL 与其存储库中提供的与 Tracer 相同类型漏洞相关的查询一起使用。CodeQL 总共报告了来自基准测试程序的 3,488 条警报。其中，我们检查了 324 条警报。

我们的实验表明，Tracer 能够有效检测 CodeQL 遗漏的重复性漏洞。在 453 个 ground truth 中，CodeQL 报告了 161 个真实警报（35.5%），而 Tracer 检测到了 253 个。

[3] https://github.com/github/codeql/blob/main/cpp/ql/src/Security/CWE/

true alarms (55.8%). Interestingly, there are no common vulnerabilities detected by both of the analyzers. There are mainly three reasons:

- Bugs can be detected by both our underlying analysis and CodeQL. However, Tracer filters out them because there is no similar trace in our signature database.
- Bugs can be detected by only CodeQL but missed by our underlying analysis. This is due to the unsoundness of our implementation. In particular, we only implemented the semantics of external libraries (e.g., `fread`, `getenv`) that are observed in our signature database. However, CodeQL supports a wider range of library models.
- Bugs can be detected by only our analysis but missed by CodeQL. We conjecture that this is mainly because of their unsound design choices as usual static bug detectors.

We agree that it is hard to make an apple-to-apple comparison between Tracer and CodeQL because they are based on different analysis frameworks, and CodeQL does not use signatures. The main goal of this comparison is to show that Tracer can accurately discover nontrivial bugs that are not detected by state-of-the-art tools.

Tracer$_{85}$ also has a lower false positive ratio than CodeQL. Among 324 inspected alarms, CodeQL reports 163 false positives (50.3%) while Tracer$_{85}$ has 71 (21.9%). Figure 14 shows a false positive reported by CodeQL. Function `readI` reads a number from an external file, and changes the endianness of the number using function `endianSwapI32`. This function involves complicated bitwise operations but does not incur an integer overflow on variable `_nPat` in `Ebwt::readIntoMemory`. Tracer$_{85}$ filters out this alarm as its score is 0.72, which is lower than the threshold. This demonstrates that Tracer's similarity score can effectively suppress false alarms compared to manually designed heuristics used in CodeQL.

5.3.3 Comparison to Devign. This section compares Tracer against a learning-based vulnerability detector, Devign. Similar to VUDDY in Section 5.3.1, we instantiated Devign to two settings: *1)* Devign$_O$ is trained with the training data provided by the authors[4], *2)* Devign$_S$ is trained with functions in our signature database. We sampled 10,000 vulnerable and 10,000 non-vulnerable functions for the training of Devign$_S$.

According to our experiments, Devign$_O$ detects only 10 vulnerabilities from the ground truths while Devign$_S$ fails to report any of them. Among the 10 true alarms, 8 of them are also detected by Tracer$_{85}$. As in previous work [47], our experiments also show that the learning-based approach is not practical to find recurring vulnerabilities. In particular, in our case, many vulnerabilities involve multiple functions. However, Devign classifies vulnerabilities in function-level granularity, that can lead to a substantial number of false negatives. Moreover, the results from the learned models are not explainable. This can significantly degrade the usability of the vulnerability detection system.

[4]https://sites.google.com/view/devign

Table 4: The precision of Tracer with different sets of features.

Threshold	W/ High-level Features TP	FP	Prec	W/O High-level Features TP	FP	Prec
0.95	154	22	0.88	154	22	0.88
0.90	192	32	0.86	198	35	0.85
0.85	253	71	0.78	256	79	0.76

5.4 RQ3: Impact of High-level Features

This section conducts a sensitivity study with different sets of features. We instantiated Tracer with two settings: similarity measures with and without high-level features. Table 4 shows the impact of high-level features.

The results show that Tracer's performance is not fundamentally limited to the choice of high-level features. When high-level features are disabled, Tracer with threshold 0.95 still reports the same set of alarms. However, the high-level features often effectively filter out typical false alarms by Tracer with lower thresholds. Tracer with thresholds 0.85 and 0.90 report 10.1% and 8.6% fewer false positives while retaining all the true alarms, when the high-level features are used.

5.5 RQ4: Scalability

This section evaluates the scalability of Tracer to large programs. We measure the whole computation time of the static analysis and similarity checking for each benchmark. Then, we report the running time of Tracer according to the size of the programs in Figure 15.

The results indicate that Tracer is scalable to large programs. On average, the static analysis takes 140.42 seconds for each package. The time spent for the feature vector construction and similarity computation is at most 2.71 seconds which is a negligible cost compared to the overall procedure. Although the analysis finishes within 20 minutes for most of the packages, some packages take considerably more time than the average. For example, hugin takes about 53 minutes. This is mainly because of the imprecision of function pointer resolution that leads to analyzing too many functions via spurious indirect calls. Another exceptional example is `gettext` that takes only 91 seconds while it comprises 982K lines of code. Despite the huge code size, the program consists of a large number of small library functions. Thus, the modular analysis can be highly parallelized.

6 RELATED WORK

Our work is inspired by a large body of research on recurring vulnerability detection. All the existing work aims at discovering recurring vulnerabilities via code reuse [23, 26, 29, 38, 47]. These approaches transform buggy code fragments within a certain boundary (e.g., functions) into various forms of vulnerability signatures such as hashes [23, 26] or dependency graphs [38, 47]. Then they search for similar representations of code fragments in the programs under investigation. On the other hand, Tracer is designed to detect vulnerabilities that share the semantically same root cause. We use a sophisticated static analysis that captures vulnerable semantics along arbitrarily long paths.

真实警报（55.8%）。有趣的是，这两款分析器并没有检测到共同的漏洞。主要原因有三：

- 我们的底层分析和 CodeQL 都能检测到 Bug，但由于我们的特征库中没有类似的 trace，Tracer 会过滤掉这些 Bug。
- 只有 CodeQL 才能检测到错误，而我们的底层分析却遗漏了这些错误。这是由于我们的实现不完善造成的。具体来说，我们只实现了在签名数据库中观察到的外部库（例如 fread、getenv）的语义。然而，CodeQL 支持更广泛的库模型。
- 仅靠我们的分析就能检测到 Bug，但 CodeQL 却会遗漏这些 Bug。我们推测，这主要是因为它们的设计选择并不完善，不像常见的静态 Bug 检测器那样完善。

我们同意，很难对 Tracer 和 CodeQL 进行完全一样的比较，因为它们基于不同的分析框架，而且 CodeQL 不使用签名。本次比较的主要目的是证明 Tracer 能够准确地发现那些最先进的工具无法检测到的复杂错误。

Tracer 的误报率也低于 CodeQL。在 324 条已检查的告警中，CodeQL 报告了 163 条误报（50.3%），而 Tracer 报告了 71 条误报（21.9%）。图 14 展示了 CodeQL 报告的一个误报。函数 readI 从外部文件读取一个数字，并使用函数 endianSwapI32 更改该数字的字节序。该函数涉及复杂的位运算，但不会导致 Ebwt::readIntoMemory 中变量 _nPat 的整数溢出。Tracer 过滤掉了该告警，因为其得分为 0.72，低于阈值。这表明，与 CodeQL 中手动设计的启发式方法相比，Tracer 的相似度得分可以有效地抑制误报。

5.3.3 与 Devign 的比较。本节将 Tracer 与基于学习的漏洞检测器 Devign 进行比较。与 5.3.1 节中的 VUDDY 类似，我们将 Devign 实例化为两种设置：1）使用作者提供的训练数据训练 Devign；2）使用我们签名数据库中的函数训练 Devign。我们采样了 10,000 个易受攻击的函数和 10,000 个非易受攻击的函数来训练 Devign。

根据我们的实验，Devign 仅从基本事实中检测到 10 个漏洞，而 Devign 未报告任何漏洞。在 10 个真实警报中，Tracer 也检测到了 8 个。与以前的工作 [47] 一样，我们的实验也表明，基于学习的方法对于查找重复出现的漏洞并不实用。尤其是在我们的案例中，许多漏洞涉及多个功能。然而，Devign 在功能级粒度上对漏洞进行分类，这可能导致大量的误报。此外，学习模型的结果无法解释。这会显著降低漏洞检测系统的可用性。

[4] https://sites.google.com/view/devign

表 4：具有不同特征集的 Tracer 的精度。

具有高级特征			不具有高级特征		
阈值	TP	FP	Prec	TP	FP	Prec
0.95	154	22	0.88	154	22	0.88
0.90	192	32	0.86	198	35	0.85
0.85	253	71	0.78	256	79	0.76

5.4 RQ3：高级特征的影响

本节对不同的特征集进行了敏感性研究。我们用两种设置实例化了 Tracer：包含和不包含高级特征的相似度度量。表 4 展示了高级特征的影响。

结果表明，Tracer 的性能并非从根本上受限于高级特征的选择。禁用高级特征时，阈值为 0.95 的 Tracer 仍会报告相同的警报集合。然而，高级特征通常能够有效地滤除阈值较低的 Tracer 的典型误报。使用高级特征时，阈值为 0.85 和 0.90 的 Tracer 报告的误报率分别降低了 10.1% 和 8.6%，同时保留了所有真实警报。

5.5 RQ4：可扩展性

本节评估 Tracer 对大型程序的可扩展性。我们测量了每个基准测试的静态分析和相似性检查的总体计算时间。然后，我们根据图 15 中的程序大小报告了 Tracer 的运行时间。

结果表明，Tracer 可扩展至大型程序。平均而言，每个软件包的静态分析耗时 140.42 秒。特征向量构建和相似度计算最多耗时 2.71 秒，与整体流程相比，这几乎可以忽略不计。虽然大多数软件包的分析在 20 分钟内完成，但某些软件包的分析时间远超平均水平。例如，hugin 耗时约 53 分钟。这主要是因为函数指针解析不精确，导致通过虚假间接调用分析了过多函数。另一个例外是 gettext，它仅耗时 91 秒，但包含 982K 行代码。尽管代码量巨大，但该程序包含大量小型库函数。因此，模块化分析可以高度并行化。

6 相关工作

我们的工作受到大量关于重复漏洞检测的研究的启发。所有现有的工作都旨在通过代码重用发现重复出现的漏洞 [23、26、29、38、47]。这些方法将特定边界内（例如函数）的错误代码片段转换为各种形式的漏洞签名，例如哈希值 [23、26] 或依赖关系图 [38、47]。然后，他们在被调查的程序中搜索代码片段的相似表示。另一方面，Tracer 旨在检测在语义上具有相同根本原因的漏洞。我们使用复杂的静态分析来捕获任意长路径上的脆弱性语义。

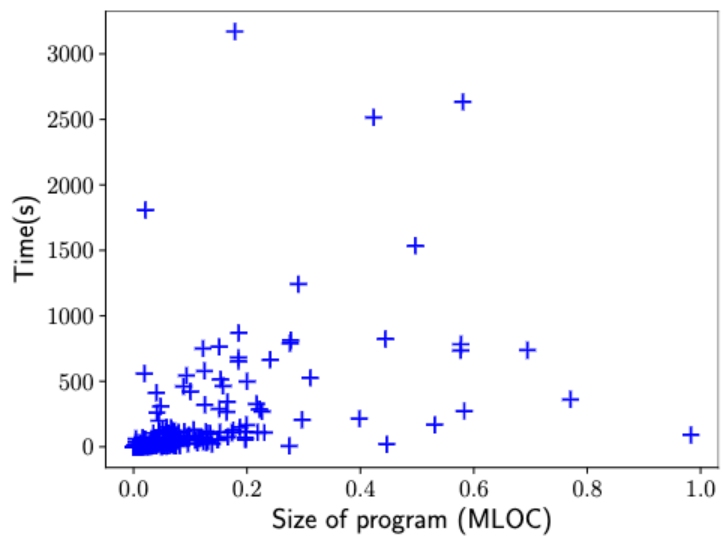


Figure 15: Running time of TRACER by program size.

Recently, several approaches have been proposed to detect vulnerabilities using learning methods [30, 31, 48]. They learn models to capture the characteristics of a variety of forms of code such as bug patches, program slices, or abstract syntax trees. Then, the learned models predict whether a slice or a function in a target program is vulnerable or not. Unlike our work, the learning-based approaches aim at general-purpose vulnerability detection. According to the literature, these approaches are not effective to detect recurring vulnerabilities [47]. This is also demonstrated by our experiments. The state-of-the-art learning-based approaches cannot detect all the recurring vulnerabilities that TRACER reports with high similarity scores.

Most of the existing static analyses that take into account code patterns highly rely on manual design [2, 3, 18]. FindBugs [3], SpotBugs [1], and ErrorProne [18] specify hundreds of human-written patterns each of which describes a specific buggy scenario. To reduce the engineering burden, CodeQL [2] introduces a query language to succinctly define bug patterns on top of their static analysis. However, it is still nontrivial for ordinary developers to write desired queries for their own purposes without static analysis expertises [32]. Instead of relying on manually written queries, TRACER automatically captures vulnerable patterns from data that provides an accessible framework for developers.

Researchers have proposed many techniques to detect code clones ranging from syntactic ones [9, 24, 40, 43, 46] to semantic ones [17, 25, 27, 41, 45]. Since their goal is to detect generally similar code fragments, they are not suitable for accurately finding recurring vulnerabilities even via code reuse [26, 47]. Instead, our work is designed to detect semantically similar vulnerabilities between two programs using a static analysis combined with a trace-based similarity measure.

Our similarity checking method can be understood as an alarm (or, analysis results, in general) ranking system for static analysis. There have been many alarm ranking methods proposed to lower the user's alarm inspection burdens. Existing approaches rank alarms by their confidence [6, 28, 35, 39], expected reactions from developers [19] or relevance to a specific commit [21]. Furthermore, recent approaches [6, 21, 39] incorporate user feedback on alarms and prioritize correlated alarms within programs. However, to our best knowledge, none of the existing work ranks alarms by similarity to a specific known vulnerability across programs.

7 CONCLUSION

We proposed TRACER, a framework for detecting semantically recurring vulnerabilities. TRACER is based on a static analysis that discovers potentially vulnerable traces in a target program. Each candidate trace is then compared with known vulnerabilities collected from various sources. Our empirical study shows that TRACER can accurately detect semantically similar vulnerabilities from a variety of open source programs. We anticipate that TRACER will allow developers to easily prevent recurring vulnerabilities without requiring static analysis expertise.

ACKNOWLEDGMENTS

This work was partly supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(No. 2021R1A5A1021944, 2021R1C1C1003876) and Institute for Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2021-0-00758, Development of Automated Program Repair Technology by Combining Code Analysis and Mining, and No.2022-0-01202, Regional strategic industry convergence security core talent training business).

REFERENCES

[1] Spotbugs. https://spotbugs.github.io, 2021.
[2] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. Ql: Object-oriented queries on relational data. In *European Conference on Object-Oriented Programming (ECOOP 2016)*, 2016.
[3] Nathaniel Ayewah, David Hovemeyer, J David Morgenthaler, John Penix, and William Pugh. Using static analysis to find bugs. *IEEE Softw.*, 25, 2008.
[4] Paul Black. Juliet 1.3 test suite: Changes from 1.2. *NIST Technical Note*, 8 2018.
[5] Cristiano Calcagno and Dino Distefano. Infer: An automatic program verifier for memory safety of c programs. In *NASA Formal Methods - Third International Symposium (NFM)*, volume 6617. Springer, 2011.
[6] Tianyi Chen, Kihong Heo, and Mukund Raghothaman. In *29th ACM Joint European Software Engineering Conferenceand Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM.
[7] CodeQL. Codeql cwe queries. https://github.com/github/codeql/tree/main/cpp/ql/src/Security/CWE, 2021.
[8] CodeQL. TaintedAllocationSize.ql. https://github.com/github/codeql/blob/main/cpp/ql/src/Security/CWE/CWE-190/TaintedAllocationSize.ql, 2021.
[9] James R Cordy and Chanchal K Roy. The NiCad clone detector. In *The 19th IEEE International Conference on Program Comprehension (ICPC 2011)*. IEEE Computer Society, 2011.
[10] The MITRE Corporation. Common vulnerabilities and exposures, 2021.
[11] The MITRE Corporation. Common weakness enumeration, 2021.
[12] Yaniv David, Nimrod Partush, and Eran Yahav. Firmup: Precise static detection of common vulnerabilities in firmware. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24-28, 2018*. ACM, 2018.
[13] Debian. Debian packages. https://packages.debian.org/sid/, 2021.
[14] Will Dietz, Peng Li, John Regehr, and Vikram S Adve. Understanding integer overflow in c/c++. In *34th International Conference on Software Engineering (ICSE 2012)*. IEEE Computer Society, 2012.
[15] Steven H. H. Ding, Benjamin C. M. Fung, and Philippe Charland. Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019.
[16] The OWASP Foundation. Attacks. https://owasp.org/www-community/attacks/, 2021.
[17] Mark Gabel, Lingxiao Jiang, and Zhendong Su. Scalable detection of semantic clones. In *30th International Conference on Software Engineering (ICSE 2008)*. ACM, 2008.
[18] Google. Error prone. https://errorprone.info, 2021.
[19] Quinn Hanam, Lin Tan, Reid Holmes, and Patrick Lam. Finding patterns in static analysis alerts: improving actionable alert ranking. In *11th Working Conference on Mining Software Repositories (MSR 2014)*. ACM, 2014.

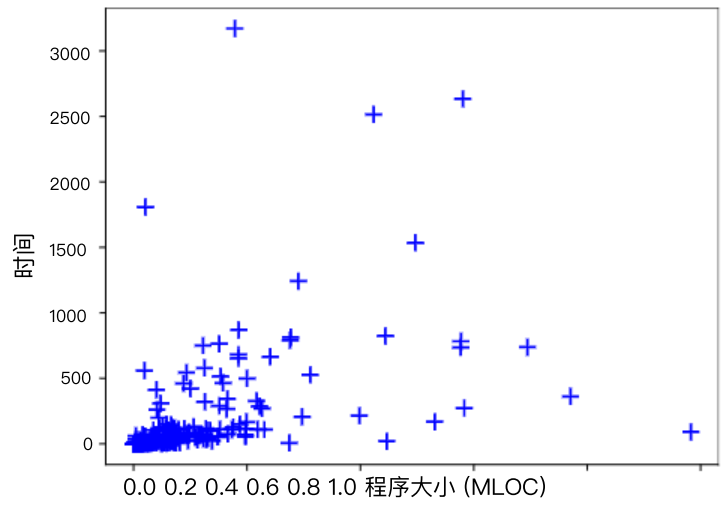


图 15：Tracer 的运行时间（按程序大小）。

最近，已经提出了几种使用学习方法检测漏洞的方法 [30，31，48]。他们学习模型来捕获各种形式代码的特征，例如错误补丁，程序切片或抽象语法树。然后，学习到的模型预测目标程序中的切片或函数是否存在漏洞。与我们的工作不同，基于学习的方法旨在进行通用漏洞检测。根据文献，这些方法对于检测重复出现的漏洞并不有效 [47]。我们的实验也证明了这一点。最先进的基于学习的方法无法检测到 Tracer 报告的具有高相似度得分的所有重复出现的漏洞。

现有的大多数考虑代码模式的静态分析高度依赖于手工设计 [2, 3, 18]。FindBugs [3]、SpotBugs [1] 和 ErrorProne [18] 指定了数百个人工编写的模式，每种模式都描述了一种特定的错误场景。为了减轻工程负担，CodeQL [2] 引入了一种查询语言，可以在静态分析的基础上简洁地定义错误模式。但是，对于普通开发人员来说，如果没有静态分析专业知识，编写用于自身目的的查询仍然不是一件容易的事 [32]。Tracer 并不依赖于手工编写的查询，而是自动从数据中捕获易受攻击的模式，从而为开发人员提供了一个可访问的框架。

研究人员已经提出了许多检测代码克隆的技术，从句法技术[9、24、40、43、46]到语义技术[17、25、27、41、45]。由于这些技术的目标是检测通常相似的代码片段，因此即使通过代码重用[26、47]，它们也不适合准确查找重复出现的漏洞。因此，我们的工作旨在使用静态分析结合基于跟踪的相似性度量来检测两个程序之间语义相似的漏洞。

我们的相似性检查方法可以理解为一种用于静态分析的警报（或一般而言，分析结果）排序系统。为了减轻用户的警报检查负担，已经提出了许多警报排序方法。现有方法根据警报的置信度 [6、28、35、39]、开发人员的预期反应 [19] 或与特定提交的相关性 [21] 对警报进行排序。此外，最近的方法 [6、21、39] 结合了用户对警报的反馈，并对程序中相关的警报进行优先级排序。然而，据我们所知，现有研究均未根据警报与程序中特定已知漏洞的相似性对警报进行排序。

7 结论

我们提出了 Tracer，一个用于检测语义重复漏洞的框架。Tracer 基于静态分析，能够发现目标程序中潜在的漏洞踪迹。之后，我们会将每个候选踪迹与从各种来源收集的已知漏洞进行比较。我们的实证研究表明，Tracer 能够准确检测出各种开源程序中语义相似的漏洞。我们预计，Tracer 将帮助开发人员轻松预防重复出现的漏洞，而无需具备静态分析方面的专业知识。

致谢

这项工作部分得到了韩国政府 (MSIT) 资助的韩国国家研究基金会 (NRF) 拨款（编号 2021R1A5A1021944、2021R1C1C1003876）和韩国政府 (MSIT) 资助的信息与通信技术规划与评估研究所 (IITP) 拨款（编号 2021-0-00758，结合代码分析和挖掘开发自动程序修复技术，以及编号 2022-0-01202，区域战略产业融合安全核心人才培训业务）的支持。

参考

[1] Spotbugs。https://spotbugs.github.io，2021 年。
[2] Pavel Avgustinov、Oege de Moor、Michael Peyton Jones 和 Max Schäfer。Ql：关系数据的面向对象查询。2016 年欧洲面向对象编程会议 (ECOOP 2016)。[3] Nathaniel Ayewah、David Hovemeyer、J David Morgenthaler、John Penix 和 William Pugh。使用静态分析查找错误。IEEE 软件，2008 年 25 日。
[4] Paul Black. Juliet 1.3 测试套件：自 1.2 版本以来的变更。NIST 技术说明，2018 年 8 月。[5] Cristiano Calcagno 和 Dino Distefano. Infer：一种自动程序验证器
用于 C 语言程序的内存安全。载于 NASA 形式化方法 – 第三届国际研讨会 (NFM) 第 6617 卷。Springer 出版社，2011 年。
[6] 陈天一，Kihong Heo，Mukund Raghothaman。在第 29 届 ACM 欧盟联合会议上绳索软件工程会议和软件工程基础研讨会（ESEC / FSE）。ACM。
[7] CodeQL。Codeql cwe 查询。https://github.com/github/codeql/tree/main/cpp/ql/src/Security/CWE，2021 年。[8] CodeQL。TaintedAllocationSize.ql。https://github.com/github/codeql/blob/main/cpp/ql/src/Security/CWE/CWE-190/TaintedAllocationSize.ql，2021 年。[9] James R Cordy 和 Chanchal K Roy。镍镉电池克隆检测器理解第 19 届 IEEE 会议 (ICPC 2011)。IEEE 计算机协会，2011 年。
[10] MITRE 公司。常见漏洞和暴露，2021 年。
[11] MITRE 公司，常见弱点枚举，2021 年。[12] Yaniv David、Nimrod Partush 和 Eran Yahav，Firmup：精确静态检测
固件中常见漏洞的概述。在第二十三届国际编程语言和操作系统架构支持会议，ASPLOS 2018，美国弗吉尼亚州威廉斯堡，2018 年 3 月 24 日至 28 日。ACM，2018。
[13] Debian. Debian 软件包. https://packages.debian.org/sid/, 2021. [14] Will Dietz, Peng Li, John Regehr, 和 Vikram S Adve. 理解整数
c/c++中的溢出。在第 34 届国际软件工程会议（ICSE 2012）上。IEEE 计算机协会，2012 年。
[15] Steven HH Ding、Benjamin CM Fung 和 Philippe Charland. Asm2vec：
提升二进制克隆搜索对代码混淆和编译器优化的静态表示鲁棒性。2019 年 IEEE 安全与隐私研讨会（SP 2019），美国加利福尼亚州旧金山，2019 年 5 月 19 日至 23 日。IEEE，2019。
[16] OWASP 基金会。攻击。https://owasp.org/www-community/attacks/，2021 年。
[17] Mark Gabel、Lingxiao Jiang 和 Zhendong Su。可扩展的语义检测
克隆。在第 30 届国际软件工程大会（ICSE 2008）上。ACM，2008 年。
[18] Google。容易出错。https://errorprone.info，2021 年。
[19] Quinn Hanam、Lin Tan、Reid Holmes 和 Patrick Lam. 在静态中寻找模式
分析警报：提升可操作警报排名。第 11 届软件存储库挖掘工作会议（MSR 2014）。ACM，2014 年。

[20] Kihong Heo, Hakjoo Oh, and Kwangkeun Yi. Machine-learning-guided selectively unsound static analysis. In *Proceedings of the 39th International Conference on Software Engineering (ICSE 2017)*. IEEE / ACM, 2017.
[21] Kihong Heo, Mukund Raghothaman, Xujie Si, and Mayur Naik. Continuously reasoning about programs using differential bayesian inference. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2019)*. ACM, 2019.
[22] IoTcube. Iotcube. https://iotcube.korea.ac.kr, 2021.
[23] Jiyong Jang, Abeer Agrawal, and David Brumley. Redebug: Finding unpatched code clones in entire os distributions. In *IEEE Symposium on Security and Privacy (S&P 2012)*. IEEE Computer Society, 2012.
[24] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stéphane Glondu. DECKARD: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE 2007)*. IEEE Computer Society, 2007.
[25] Heejung Kim, Yungbum Jung, Sunghun Kim, and Kwangkeun Yi. MeCC: memory comparison-based clone detector. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE 2011)*. ACM, 2011.
[26] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. VUDDY: A scalable approach for vulnerable code clone discovery. In *IEEE Symposium on Security and Privacy (S&P 2017)*. IEEE Computer Society, 2017.
[27] Raghavan Komondoor and Susan Horwitz. Using slicing to identify duplication in source code. In *Proceedings of 8th International Static Analysis Symposium (SAS 2001)*, volume 2126. Springer, 2001.
[28] Ted Kremenek and Dawson R Engler. Z-ranking: Using statistical analysis to counter the impact of static analysis approximations. In *Proceedings of 10th International Static Analysis Symposium (SAS 2003)*, volume 2694. Springer, 2003.
[29] Jingyue Li and Michael D Ernst. Cbcd: Cloned buggy code detector. In *34th International Conference on Software Engineering (ICSE 2012)*. IEEE Computer Society, 2012.
[30] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. VulPecker: an automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC 2016)*. ACM, 2016.
[31] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. VulDeePecker: A deep learning-based system for vulnerability detection. In *25th Annual Network and Distributed System Security Symposium (NDSS 2018)*. The Internet Society, 2018.
[32] Ziyang Li, Aravind Machiry, Binghong Chen, Mayur Naik, Ke Wang, and Le Song. Arbitrar : User-guided api misuse detection. *IEEE Symposium on Security and Privacy (S&P 2021)*, 2021.
[33] Cristina V Lopes, Petr Maj, Pedro Martins, Vaibhav Saini, Di Yang, Jakub Zitny, Hitesh Sajnani, and Jan Vitek. Déjàvu: a map of code duplicates on github. *Proc. ACM Program. Lang.*, 1, 2017.
[34] Antonio Nappa, Richard Johnson, Leyla Bilge, Juan Caballero, and Tudor Dumitras. The attack of the clones: A study of the impact of shared code on vulnerability patching. In *IEEE Symposium on Security and Privacy (S&P 2015)*, 2015.
[35] Damien Octeau, Somesh Jha, Matthew Dering, Patrick D. McDaniel, Alexandre Bartel, Li Li, Jacques Klein, and Yves Le Traon. Combining static analysis with probabilistic models to enable market-scale android inter-component analysis. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2016.
[36] OWASP. Buffer overflow via environment variables. https://owasp.org/www-community/attacks/Buffer_Overflow_via_Environment_Variables, 2021.
[37] Soyeon Park, Wen Xu, Insu Yun, Daehee Jang, and Taesoo Kim. Fuzzing javascript engines with aspect-preserving mutation. In *IEEE Symposium on Security and Privacy (S&P 2020)*. IEEE, 2020.
[38] Nam H Pham, Tung Thanh Nguyen, Hoan Anh Nguyen, and Tien N Nguyen. Detection of recurring software vulnerabilities. In *25th IEEE/ACM International Conference on Automated Software Engineering (ASE 2010)*. ACM, 2010.
[39] Mukund Raghothaman, Sulekha Kulkarni, Kihong Heo, and Mayur Naik. User-guided program reasoning using bayesian inference. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. ACM, 2018.
[40] Hitesh Sajnani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K Roy, and Cristina V Lopes. Sourcerercc: scaling code clone detection to big-code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE 2016)*. ACM, 2016.
[41] Abdullah Sheneamer and Jugal Kalita. Semantic clone detection using machine learning. In *15th IEEE International Conference on Machine Learning and Applications (ICMLA 2016)*. IEEE Computer Society, 2016.
[42] Maddie Stone. Déjà vu-lnerability. https://googleprojectzero.blogspot.com/2021/02/deja-vu-lnerability.html, 2021.
[43] Pengcheng Wang, Jeffrey Svajlenko, Yanzhao Wu, Yun Xu, and Chanchal K Roy. CCAligner: a token based large-gap clone detector. In *Proceedings of the 40th International Conference on Software Engineering (ICSE 2018)*. ACM, 2018.
[44] Xi Wang, Haogang Chen, Zhihao Jia, Nickolai Zeldovich, and M Frans Kaashoek. Improving integer security for systems with kint. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2012)*. USENIX Association, 2012.
[45] Huihui Wei and Ming Li. Positive and unlabeled learning for detecting software functional clones with adversarial training. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI 2018)*. ijcai.org, 2018.
[46] Martin White, Michele Tufano, Christopher Vendome, and Denys Poshyvanyk. Deep learning code fragments for code clone detection. In *Proceedings of the 31st IEEE/ACM International Conference on AutomatedSoftware Engineering (ASE 2016)*. ACM, 2016.
[47] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, and Wenchang Shi. MVP: Detecting vulnerabilities using patch-enhanced vulnerability signatures. In *29th USENIX Security Symposium (USENIX Security 2020)*. USENIX Association, 2020.
[48] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Neural Information Processing Systems 2019 (NeurIPS 2019)*, 2019.

[20] Kihong Heo、Hakjoo Oh 和 Kwangkeun Yi。机器学习引导选择性
不健全的静态分析。刊于第 39 届国际软件工程会议 (ICSE 2017) 论文集。IEEE / ACM，2017 年。
[21] Kihong Heo、Mukund Raghotaman、Xujie Si 和 Mayur Naik。不断地
使用差分贝叶斯推理进行程序推理。刊登于第 40 届 ACM SIGPLAN 编程语言设计与实现会议 (PLDI 2019) 论文集。ACM，2019 年。

[22] 物联网立方体。物联网。 https://iotcube.korea.ac.kr，2021 年。
[23] Jiyong Jang、Abeer Agrawal 和 David Brumley。Redebug：查找未修补的
整个操作系统发行版中的代码克隆。在 IEEE 安全与隐私研讨会 (S&P 2012) 上。IEEE 计算机协会，2012 年。
[24] 蒋凌霄，Ghassan Misherghi，苏振东，Stéphane Glondu。
DECKARD：可扩展且准确的基于树的代码克隆检测。第 29 届
国际软件工程会议(ICSE 2007)。IEEE 计算机协会，2007 年。

[25] Heejung Kim，Yungbum Jung，Sunghun Kim，和 Kwangkeun Yi。 MeCC：记忆-
基于 ory 比较的克隆检测器。刊登于第 33 届国际软件工程会议（ICSE 2011）论文集。ACM，2011 年。
[26] Seulbae Kim、Seunghoon Woo、Heejo Lee 和 Hakjoo Oh。VUDDY：可扩展的
一种用于发现易受攻击代码克隆的方法。在 IEEE 安全与隐私研讨会 (S&P 2017) 上。IEEE 计算机学会，2017 年。
[27] Raghavan Komondoor 和 Susan Horwitz. 使用切片识别重复
在源代码中。载于《第八届国际静态分析研讨会论文集》（SAS 2001），第 2126 卷。Springer 出版社，2001 年。
[28] Ted Kremenek 和 Dawson R Engler. Z 排名：使用统计分析来
抵消静态分析近似的影响。在第十届
国际静态分析研讨会 (SAS 2003)，第 2694 卷。Springer，2003 年。[29] Jingyue Li 和 Michael D Ernst。Cbcd：克隆错误代码检测器。第 34 届
国际软件工程会议(ICSE 2012)。IEEE 计算机协会，2012 年。

[30] 李震，邹德庆，徐守怀，金海，齐汉超，胡杰。 VulPicker：基于代码相似性分析的自动化漏洞检测系统。
在第 32 届计算机安全应用年会（ACSAC 2016）论文集上。ACM，2016 年。

[31] 李震，邹德庆，徐守怀，欧新宇，金海，王素娟，邓志军，
以及 Yuyi Zhong。VulDeePecker：基于深度学习的漏洞检测系统
检测。在第 25 届年度网络和分布式系统安全研讨会（NDSS 2018）上。互联网协会，2018 年。
[32] 李紫阳，Aravind Machiry，陈丙红，Mayur Naik，王科，宋乐。
Arbitrar：用户引导的 API 滥用检测。IEEE 安全与隐私研讨会 (S&P 2021)，2021 年。

[33] Cristina V Lopes、Petr Maj、Pedro Martins、Vaibhav Saini、Di Yang、Jakub Zitny、Hitesh Sajnani 和 Jan Vitek。 Déjàvu：github 上的代码重复图。过程。
ACM Program. Lang.，1，2017。
[34] Antonio Nappa、Richard Johnson、Leyla Bilge、Juan Caballero 和 Tudor Du-
mitras。克隆人的攻击：关于共享代码对
漏洞修补。在 IEEE 安全与隐私研讨会 (S&P 2015) 中，
2015.
[35]达米安·奥托、萨梅什·贾、马修·德林、帕特里克·D·麦克丹尼尔、亚历山大
Bartel, Li Li, Jacques Klein 和 Yves Le Traon。将静态分析与概率模型相结合，实现市场规模的 Android 组件间分析。发表于第 43 届 ACM SIGPLAN-SIGACT 编程语言原理研讨会 (POPL) 论文集。ACM，2016 年。

[36] OWASP. 通过环境变量进行缓冲区溢出。
https://owasp.org/wwwcommunity/attacks/Buffer_Overflow_via_Environment_Variables，2021 年。[37] Soyeon Park、Wen Xu、Insu Yun、Daehee Jang 和 Taesoo Kim。
JavaScript 具有保留语义变的引擎。在 IEEE 安全与隐私研讨会 (S&P 2020) 上。IEEE，2020 年。
[38] Nam H Pham、Tung Thanh Nguyen、Hoan Anh Nguyen 和 Tien N Nguyen。
检测重复出现的软件漏洞。在第 25 届 IEEE/ACM 自动软件工程国际会议（ASE 2010）上。ACM，2010 年。
[39] Mukund Raghotaman、Sulekha Kulkarni、Kihong Heo 和 Mayur Naik。用户-
使用贝叶斯推理进行引导程序推理。刊于第 39 届 ACM SIGPLAN 编程语言设计与实现会议 (PLDI 2018) 论文集。ACM，2018 年。

[40] Hitesh Sajnani、Vaibhav Saini、Jeffrey Svajlenko、Chanchal K Roy 和 Cristina V Lopes。Sourcerercc：将代码克隆检测扩展到大代码。在
第 38 届国际软件工程会议 (ICSE 2016)。ACM，2016 年。[41] Abdullah Sheneamer 和 Jugal Kalita。基于机器
学习。第十五届 IEEE 机器学习与应用国际会议（ICMLA 2016）。IEEE 计算机协会，2016 年。
[42] Maddie Stone. 似曾相识的感觉。
https://googleprojectzero.blogspot.com/2021/02/deja-vu-lnerability.html，2021 年。
[43]王鹏程，Jeffrey Svajlenko，吴艳照，徐云，Chanchal K Roy。
CCAligner：基于 token 的大间隙克隆检测器。发表于第 40 届国际软件工程会议（ICSE 2018）论文集。ACM，2018 年。
[44] 王曦，陈浩刚，贾志浩，Nickolai Zeldovich，M Frans Kaashoek。
使用 kint 提高系统的整数安全性。在第十届 USENIX 研讨会上
操作系统设计与实现 (OSDI 2012)。USENIX 协会，2012 年。

[45] Huihui Wei 和 Ming Li. 用于检测软件的正向和无标记学习
具有对抗性训练的功能克隆。刊于第 27 届国际人工智能联合会议（IJCAI 2018）论文集。ijcai.org，2018 年。
[46] 马丁·怀特、米歇尔·图法诺、克里斯托弗·旺多姆和丹尼斯·波希瓦尼克。
用于代码克隆检测的深度学习代码片段。刊于第 31 届 IEEE/ACM 自动软件工程国际会议 (ASE 2016) 论文集。ACM，2016 年。

[47] 肖阳，陈碧环，余晨东，徐正子，袁子木，李峰，
刘兵红，刘洋，霍伟，邹伟，石文昌。MVP：使用补丁增强的漏洞签名检测漏洞。第 29 届 USENIX 安全研讨会（USENIX Security 2020）。USENIX 协会，2020 年。

[48] 周亚勤，刘尚清，萧敬凯，杜晓宁，刘阳。设计：
通过学习全面的程序语义来有效地识别漏洞
通过图神经网络进行运动控制。载于《神经信息处理系统 2019》（NeurIPS 2019），2019 年。